HW7 COSOLIDATED REPORT

Katie Davenport
CS 6650: Scalable Distributed Systems
Homework 7: When Your Startup's Flash Sale Almost Failed

Part II

For phases I and III we ran the following Locust tests

- Normal Ops: 5 concurrent users, 1 user/second ramp-up, ~30 second test
- Flash Sale: 20 concurrent users, 10 users/second ramp-up, ~60 second test

Both tests used 1 Locust worker and used the ALB endpoint as the host.

Phase I: Build Concurrent Systems

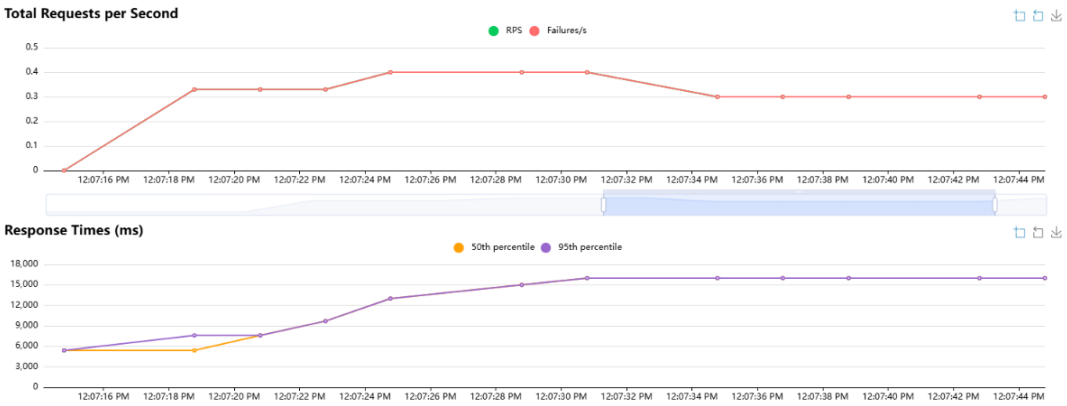
Phase I implements synchronous order processing with a 3-second delay to verify the payment for each order. Customers experience a poor checkout experience in both the normal operations and flash sale scenarios, as shown in the Locust and AWS statistics below. Because the payment step blocks the request, the system cannot keep up with incoming traffic, causing requests to fail or time out. The response times are significantly longer and increase over the course of the trial in the flash sale scenario, reflecting that incoming requests are backing up while waiting for the payment step to complete.

AWS metrics show that CPU and memory remained far below capacity in both scenarios. This indicates that the bottleneck was the synchronous payment-processing design rather than compute or memory. Because requests were blocked waiting for the 3-second payment step, throughput was minimal.

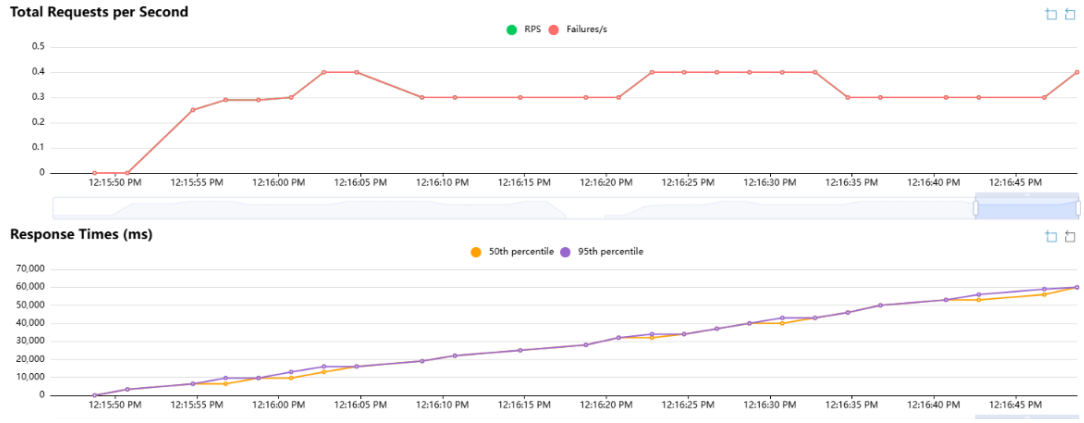
Locust Statistics

	# Requests	# Fails	Average(ms)	Median(ms)	95%ile(ms)	Current RPS
Normal Ops	18	18	13,490	15,771	16,000	0.3
Flash Sale	26	26	38,928	40,000	60,000	0.3

Locust Chart: Normal Operations



Locust Chart: Flash Sale



AWS Statistics: ECS

	CPU Utilization			Memory Utilization		
	Min	Avg	Max	Min	Avg	Max
Normal Ops	0	0.3	4.2	8.4	8.4	8.4
Flash Sale	0	0.6	4.4	8.4	8.4	8.4

AWS Statistics: ALB

	Target Response Time (sec)	Requests	Target 5XXs
Normal Ops	11.9	18	N/A
Flash Sale	40.2	32.3	N/A

AWS ALB Charts: Normal (~16:26) and Flash Sale (~16.28)



Phase 2: Analyze the Bottleneck

The maximum throughput does not increase with the number of customers. Rather, it can be calculated using the payment processor speed:

Max throughput = 1/3 order per second

The flash sale demand = 20 orders / second.
The number of orders lost per second = 20 – 1/3 = 19 2/3.
In other words, only 1 out of 60 requests per 3-second period is processed immediately.

Phase 3: The Async Solution

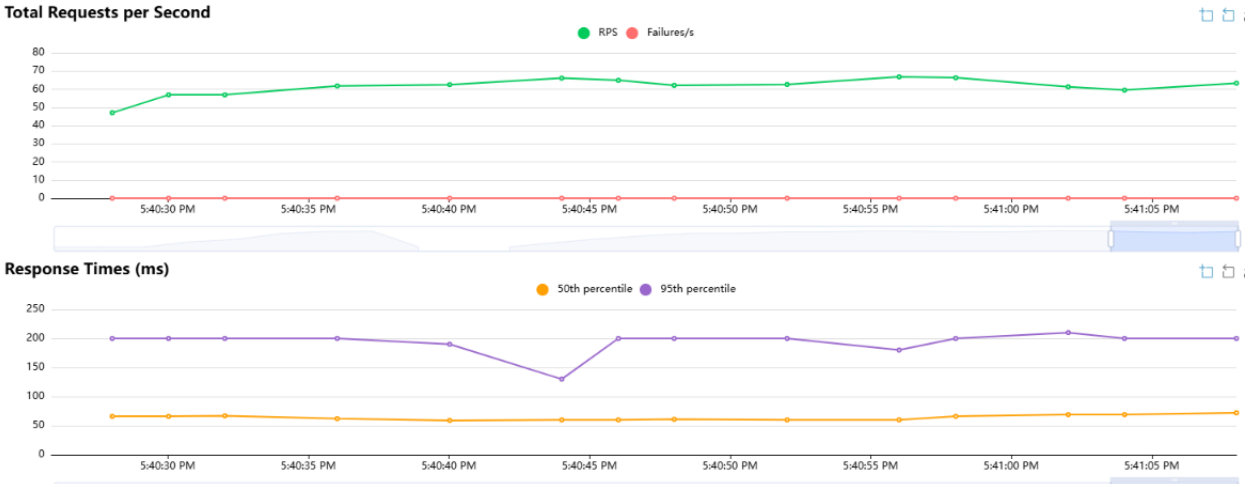
In this phase we replace synchronous payment processing with an asynchronous solution using SNS and SQS. The API immediately accepts orders and publishes them to a queue, returning a 202 Accepted response to the customer while background ECS workers process payments independently. The order processor continuously polls SQS using long polling, waiting up to 20 seconds for messages and retrieving up to 10 messages per request. A goroutine is created to process the order for each message.

This solution significantly improves performance. There are zero failures, much higher throughput, and far lower latency compared to the synchronous design. The hw7-orders service now has higher CPU utilization because it is successfully handling many more incoming requests. In contrast, the order processor service consumes less CPU but more memory since it continuously polls SQS and manages queued messages while processing orders in the background.

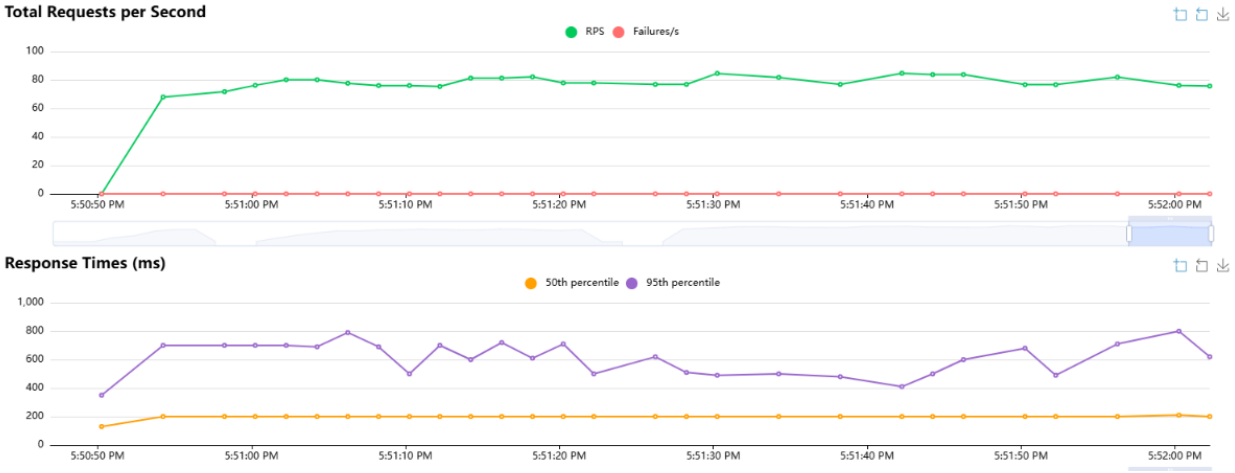
Locust Statistics

	# Requests	# Fails	Average(ms)	Median(ms)	95%ile(ms)	Current RPS
Normal Ops	2,910	0	80	66	200	63.3
Flash Sale	5,809	0	249	200	600	75.7

Locust Chart: Normal Operations



Locust Chart: Flash Sale



AWS Statistics: ECS

	CPU Utilization			Memory Utilization		
	Min	Avg	Max	Min	Avg	Max
hw7-orders						
Normal Ops	0	22.4	99.6	17.0	23.3	26.2
Flash Sale	0	23.0	99.8	12.7	25.9	33.3
Hw7-orders-processor						
Normal Ops	0	9.2	28.9	0	51.9	96.6
Flash Sale	0	13.5	49.8	0	58.9	94.5

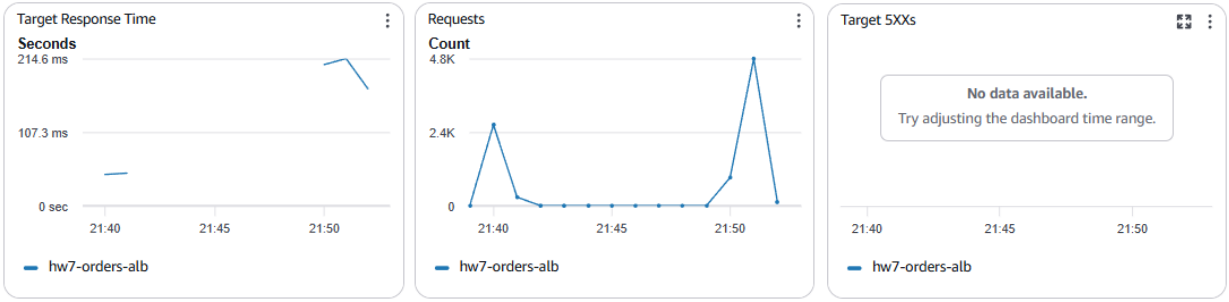
Notes:

- The hw7-orders cluster, the API behind the ALB, reflects incoming traffic.
- The hw7-orders-processor, which contains the order processor, indicates how quickly the queue is being drained.

AWS Statistics: ALB

	Target Response Time (ms)	Requests	Target 5XXs
Normal Ops	48	2,638	N/A
Flash Sale	215	4,795	N/A

AWS ALB Charts: Normal (~21:40) and Flash Sale (~21:50)



Approximate Number of Messages Visible



Phase 4: The Async Solution

CloudWatch showed a peak of 925 visible messages in the queue during the test (see above).

Order acceptance rate: ~60 messages / second
Single worker processing rate: = 0.33 messages / second
Queue growth rate: = Incoming orders – processing capacity
= 60 messages / sec – 0.33 messages / sec = 59.67 messages / second

Time to clear backlog: 925 messages at peak backlog with a 0.33 messages / second processing rate
= 925 messages / (0.33 messages / second) = 2,775 seconds
= 2,775 seconds * 1 minute / 60 seconds = 46.25 minutes to process

It would take about 46 minutes to clear the backlog with a single worker.

Phase 5: Scale Your Workers

For this phase we ran the flash sale test (20 concurrent users, 10 users/second ramp-up, ~60 second test) with 5, 20, and 100 workers. The processing rates should be:

- 5 workers: 5 * 0.33 orders / second = 1.67 orders/sec
- 20 workers: 20 * 0.33 orders / second = 6.67 orders/sec
- 100 workers: 100 * 0.33 orders / second = 33.33 orders/sec

Following are the calculations for the estimated time to drain the queue:

Workers	Processing Rate (orders / sec)	Approx Peak Visible Queue Depth	Time to Drain
5	1.67	5,400	5,400 / 1.67 * 1 min / 60 sec = 54 min
20	6.67	3,600	3,600 / 6.67 * 1 min / 60 sec = 9 min
100	33.33	2,300	2,300 / 33.33 * 1 min / 60 sec = 1 min

Resource Utilization (Note: hw7-orders-processor only)

	CPU Utilization			Memory Utilization		
Workers	Min	Avg	Max	Min	Avg	Max
5	0	37.2	93.5	0	48.7	96.0
20	0	9.1	21.2	8.8	37.2	81.6
100	0	11.4	29.1	8.8	24.5	69.6

Below is the Cloudwatch graph of the Approximate Number of Messages Visible
(Test timing: 5 workers at ~23:55, 20 workers at ~ 00:10, 100 workers at ~00:20)



Analysis questions:

What's the minimum workers needed to prevent queue buildup at 60 orders/second?
180 workers: 180 * 0.33 orders / second = 60 orders / sec

How many times more orders did your asynchronous approach accept compared to your synchronous approach?
There were 0 successful orders for synchronous tests since requests timed out while waiting for the 3-second payment step. In contrast, the asynchronous design accepted thousands of requests with zero failures. This is because orders were immediately placed on the queue and processed in the background. This represents a dramatic improvement in acceptance rate and system responsiveness.

What causes queue buildup and how do you prevent it?
Queue buildup occurs when orders arrive faster than workers can process them. Even though the API can accept requests quickly, the payment step still takes 3 seconds per order, so messages accumulate in the queue when processing capacity is insufficient. Queue buildup can be prevented by increasing processing capacity, either by adding more worker goroutines or by scaling out the processor service to run multiple ECS tasks that consume messages from the same queue.

When would you choose sync vs async in production?
Synchronous processing is appropriate when operations are fast and the user needs an immediate response. Asynchronous processing is better for longer-running tasks or bursty workloads, where requests can be accepted quickly and processed in the background without blocking the user.

Part III: What If You Didn't Need Queues?

How often do cold starts occur? (First request, after ~5+ minutes idle)
I had two log streams and they each had a cold start during the first invocation:

- First log stream with Init Duration: 74.01 ms
- Second log stream with Init Duration: 71.89 ms

All of the other invocations shown are warm starts. Based on the logs, cold starts occurred only when Lambda created a new execution environment. Note that the longest idle gap in the logs was less than 2 minutes.

What's the overhead? (73ms on 3000ms = 2.4% impact)
The overhead for the cold starts averaged 73 ms. This represents approximately 2.4% of the roughly 3 second processing duration.

Does this matter for 3-second payment processing?
The impact is minimal for this workload.

The Decision
How often did cold starts occur? Every few minutes? Every request?
As discussed above, cold starts occurred only on the first invocation of a new execution environment. In the logs, there were 2 cold starts across 9 invocations. The remaining requests were warm starts.

Is the cost advantage compelling? For 10,000 orders/month, Lambda costs $0 vs ECS \$17 (FREE within free tier!)
Yes. For a startup handling around 10,000 orders per month, Lambda would remain within the free tier compared to $17/month for two always-running ECS tasks.

Can you accept losing SQS guarantees? Messages get 2 retries, then disappear
In this case we are processing orders so it may not be acceptable to lose SQS guarantees and risk message loss.

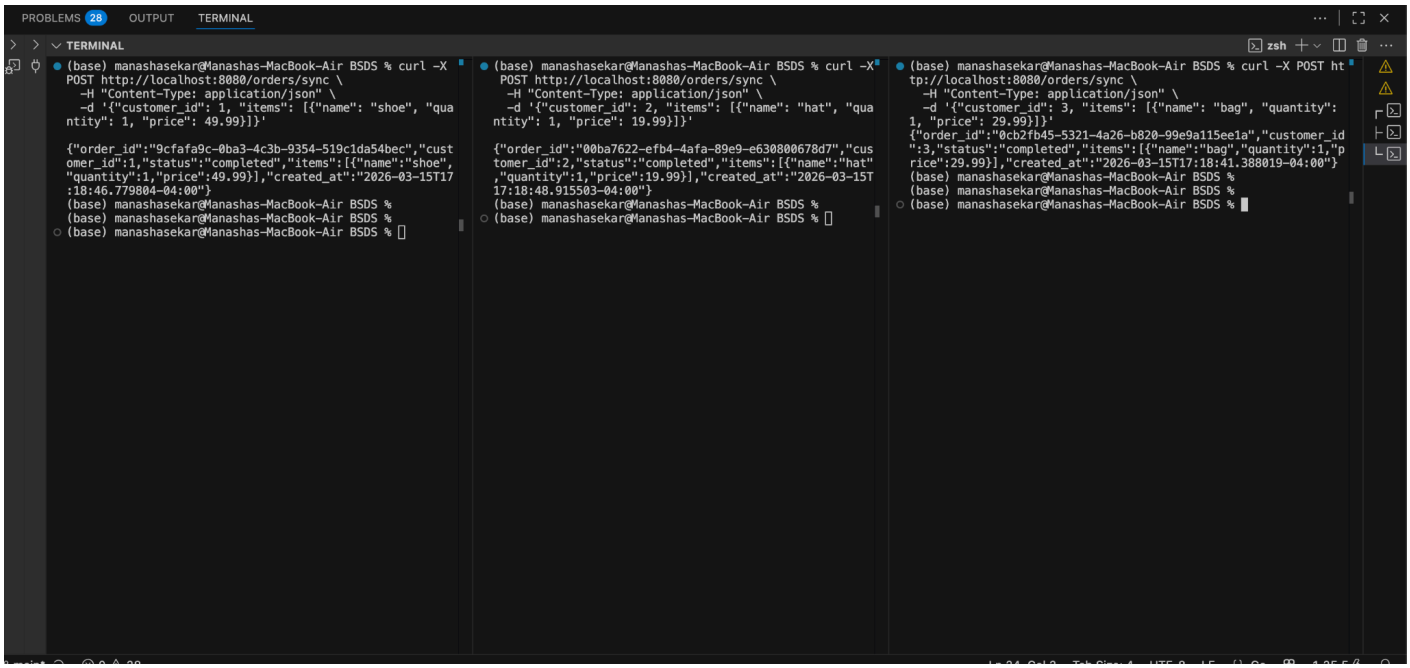
Scale consideration: Lambda stays FREE until 267K orders/month
This volume of orders seems quite large for a startup.

Name: Manasha Sekar

Homework 7: When Your Startup's Flash Sale Almost Failed

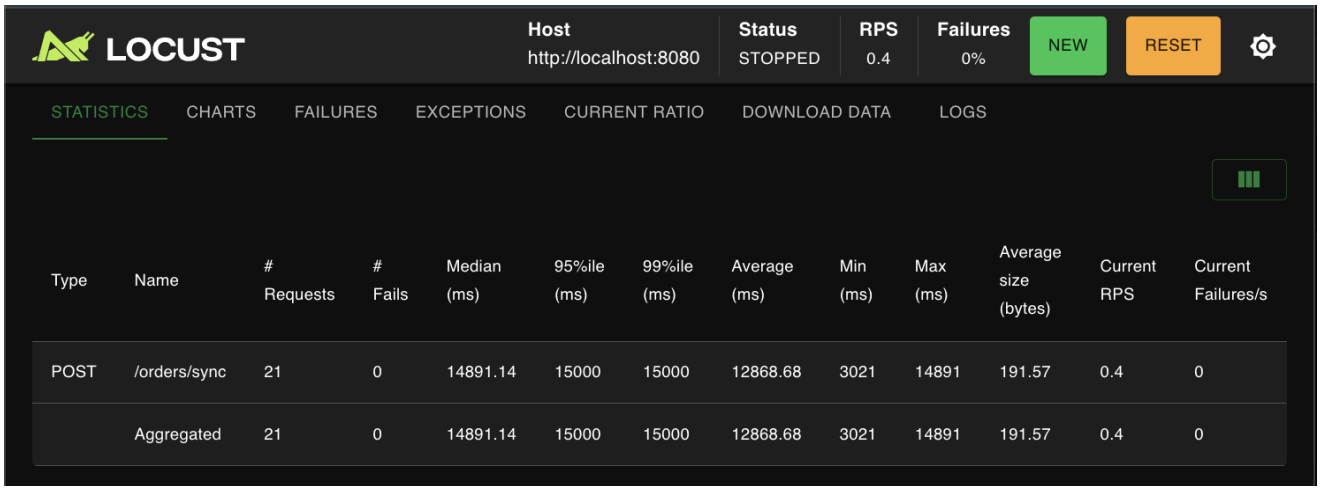
Phase 1

Local curl test:



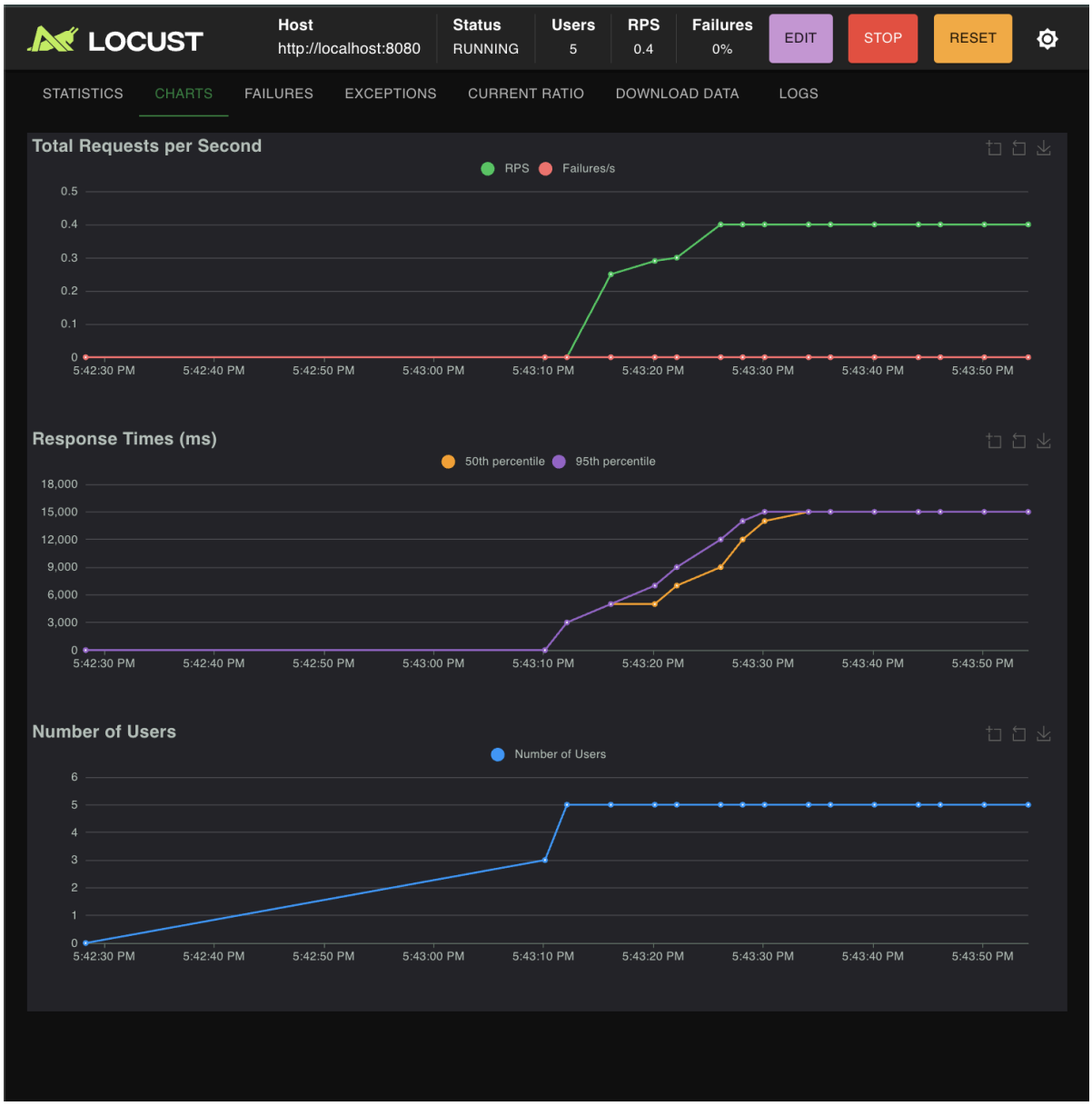
I sent the tests simultaneously but the responses took time to reach, even when requests are sent parallely is processed in sequence and then the results are returned.

Normal Test: 5 users for 30 secs.



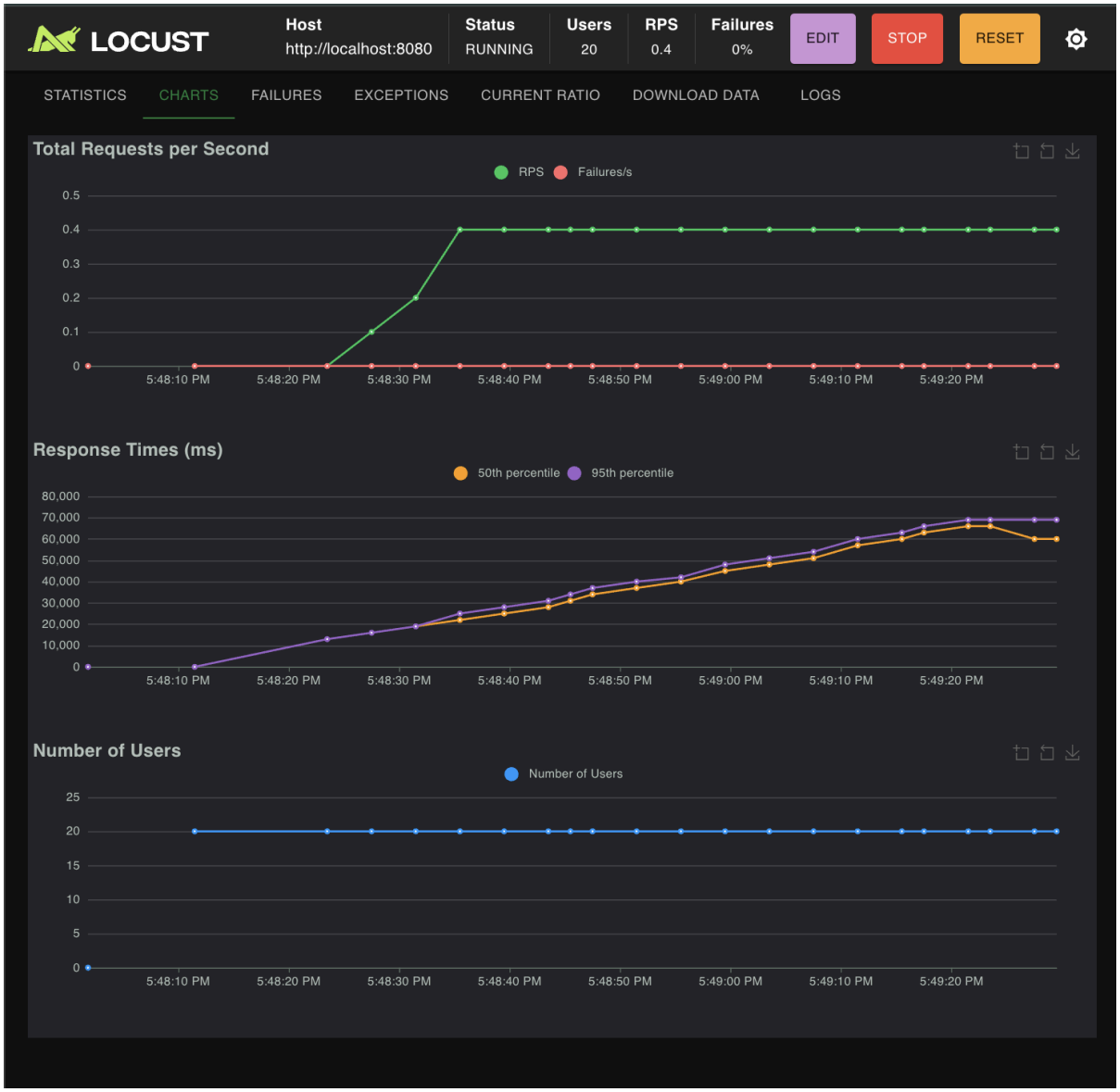
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders/sync	21	0	14891.14	15000	15000	12868.68	3021	14891	191.57	0.4	0
	Aggregated	21	0	14891.14	15000	15000	12868.68	3021	14891	191.57	0.4	0

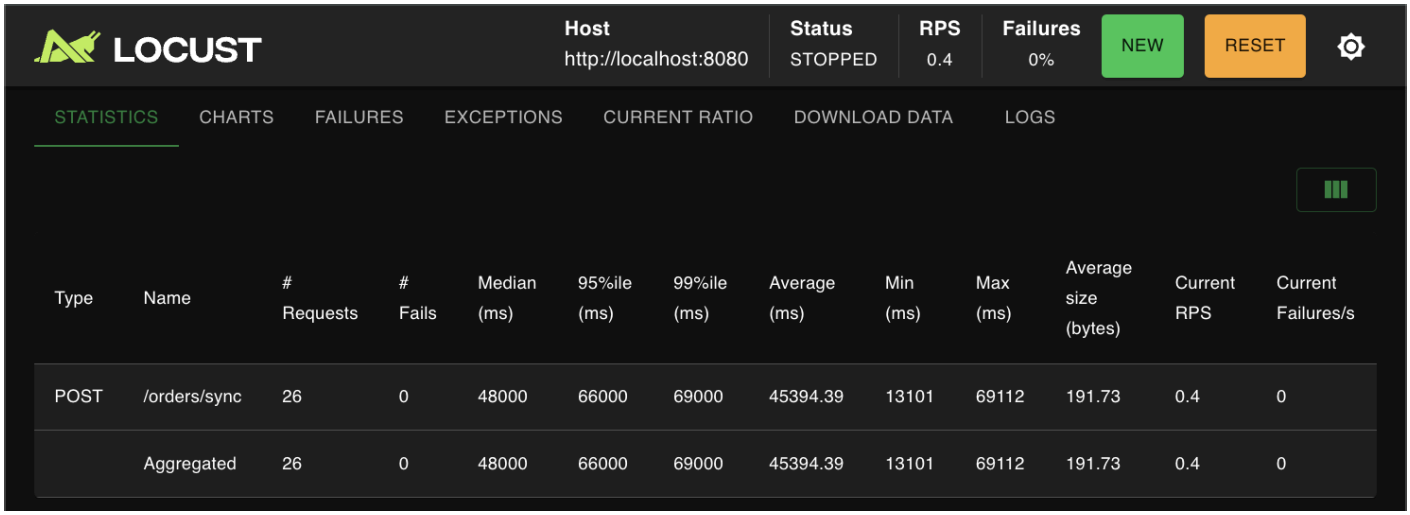
Name: Manasha Sekar



Name: Manasha Sekar

Flash sale for 20 users 60 secs.





Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders/sync	26	0	48000	66000	69000	45394.39	13101	69112	191.73	0.4	0
	Aggregated	26	0	48000	66000	69000	45394.39	13101	69112	191.73	0.4	0

Name: Manasha Sekar

Comparison:

Test	Users	RPS	Failures	95th percentile
Normal	5	0.4	0%	~15,000ms
Flash Sale	20	0.4	0%	~70,000ms

The system does not crashes it just makes the customers wait forever!

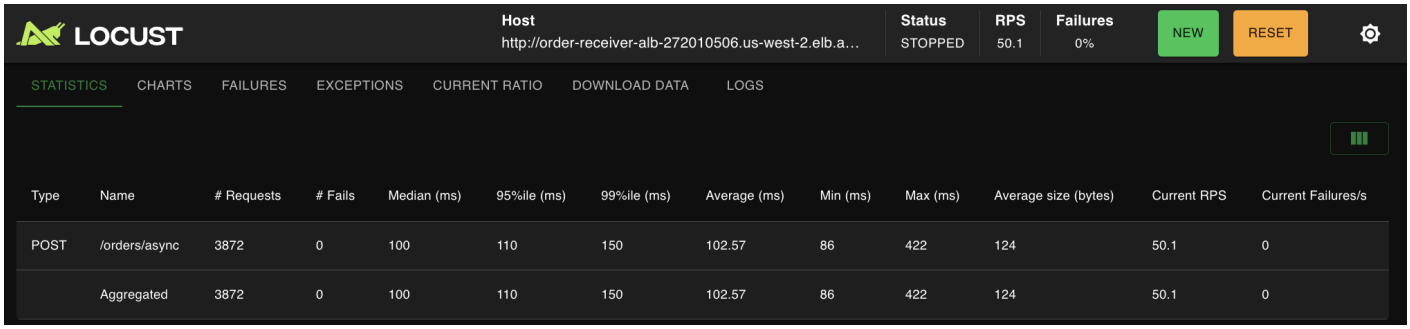
Phase 2: Analyze the Bottleneck

Payment processor speed: 1 order per 3 seconds
With 20 concurrent customers: Maximum throughput = 0.33 orders/second
Flash sale demand: 20 orders/second
Orders lost per second: 19.67 orders

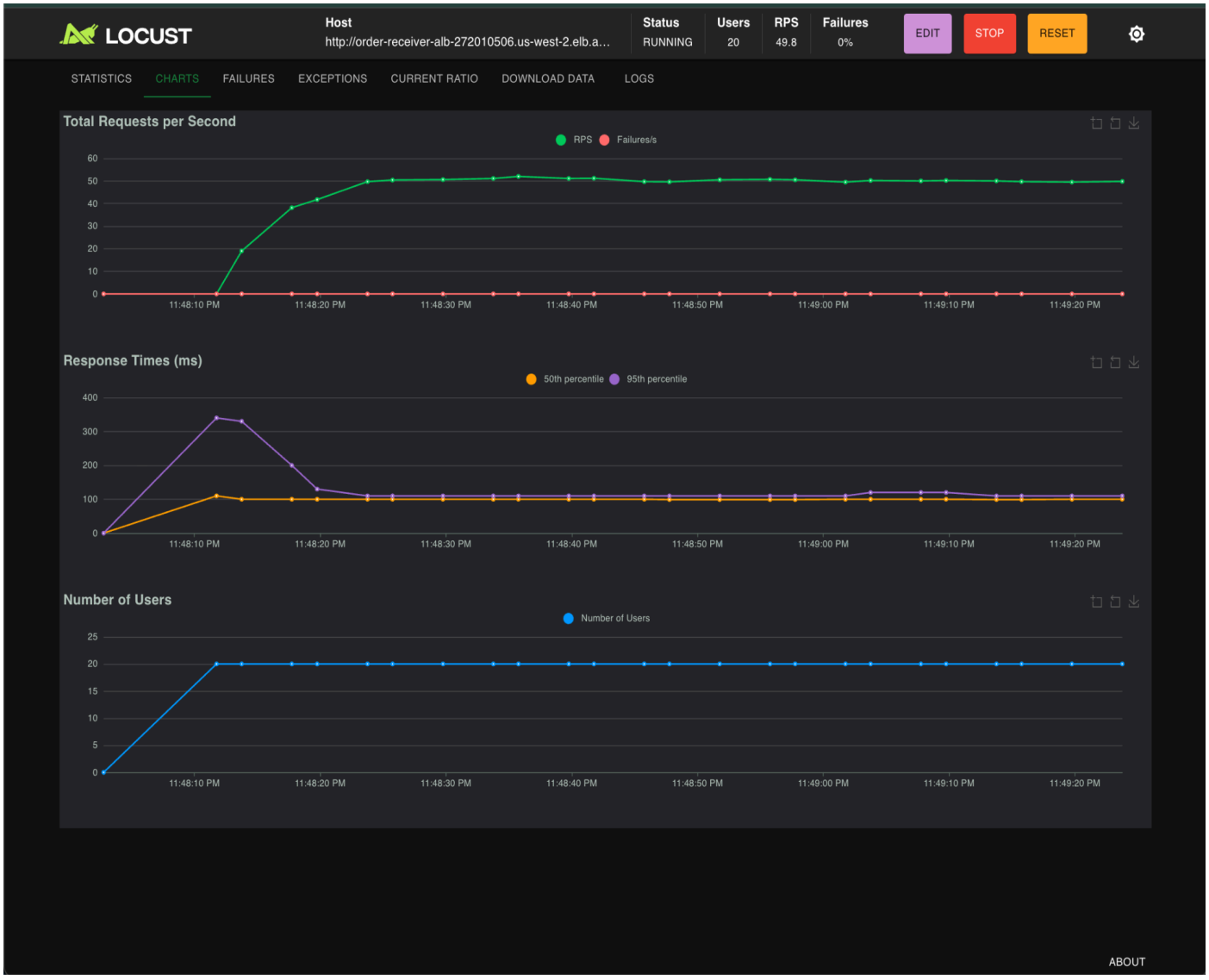
Name: Manasha Sekar

Phase 3: The Async Solution

Worker:1

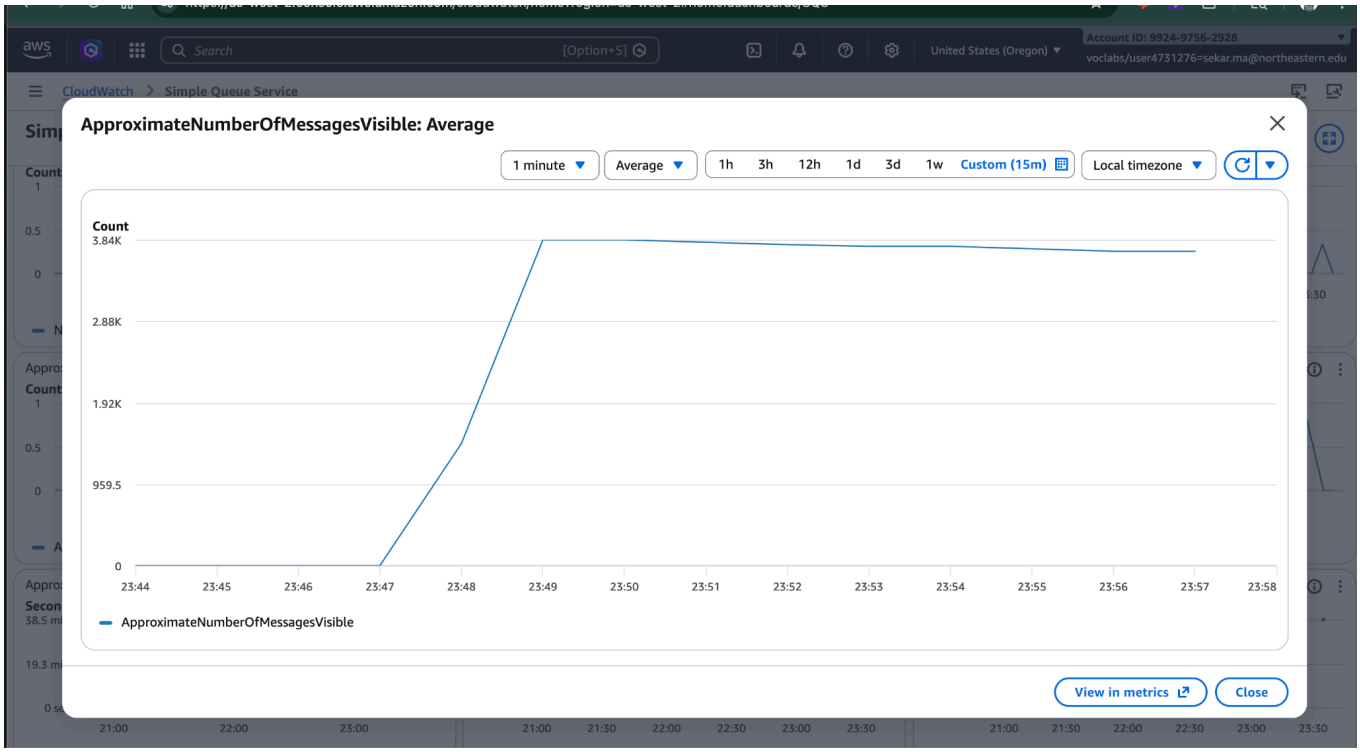


Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders/async	3872	0	100	110	150	102.57	86	422	124	50.1	0
	Aggregated	3872	0	100	110	150	102.57	86	422	124	50.1	0



Name: Manasha Sekar

Cloud Watch:



Test Configuration: 20 users, 60 seconds, spawn rate 10/s

Metric	Sync (Flash Sale)	Async (Flash Sale)
RPS	0.4	49.9
Failures	0%	0%
Avg Response Time	~45,000ms	~102ms
Orders Accepted	26	3,872

The async endpoint accepted **3,872 orders** compared to just **26** with the synchronous approach — roughly **149x more orders** in the same time window. Response times dropped from an average of 45 seconds down to 102ms, meaning customers received an immediate acknowledgment instead of waiting for payment verification to complete.

This is the core trade-off of async processing: the API returns a `202 Accepted` instantly and delegates the actual work to background workers. Customers are never blocked, and the system can absorb any burst of traffic regardless of how slow the downstream payment processor is.

Name: Manasha Sekar

Phase 4

Analysis:

- Order acceptance rate: ~50/second
- Single worker processing rate: 1 order per 3 seconds = 0.33/second
- Queue growth rate: 49.67 messages/second
- Time to clear backlog: ~192 minutes

Observations

While the async endpoint successfully accepted all 3,872 orders with 0% failures, the single background worker could only process one order every 3 seconds. During the 60-second flash sale, the queue grew at roughly 49.67 messages per second, peaking at approximately 3,840 visible messages in CloudWatch.

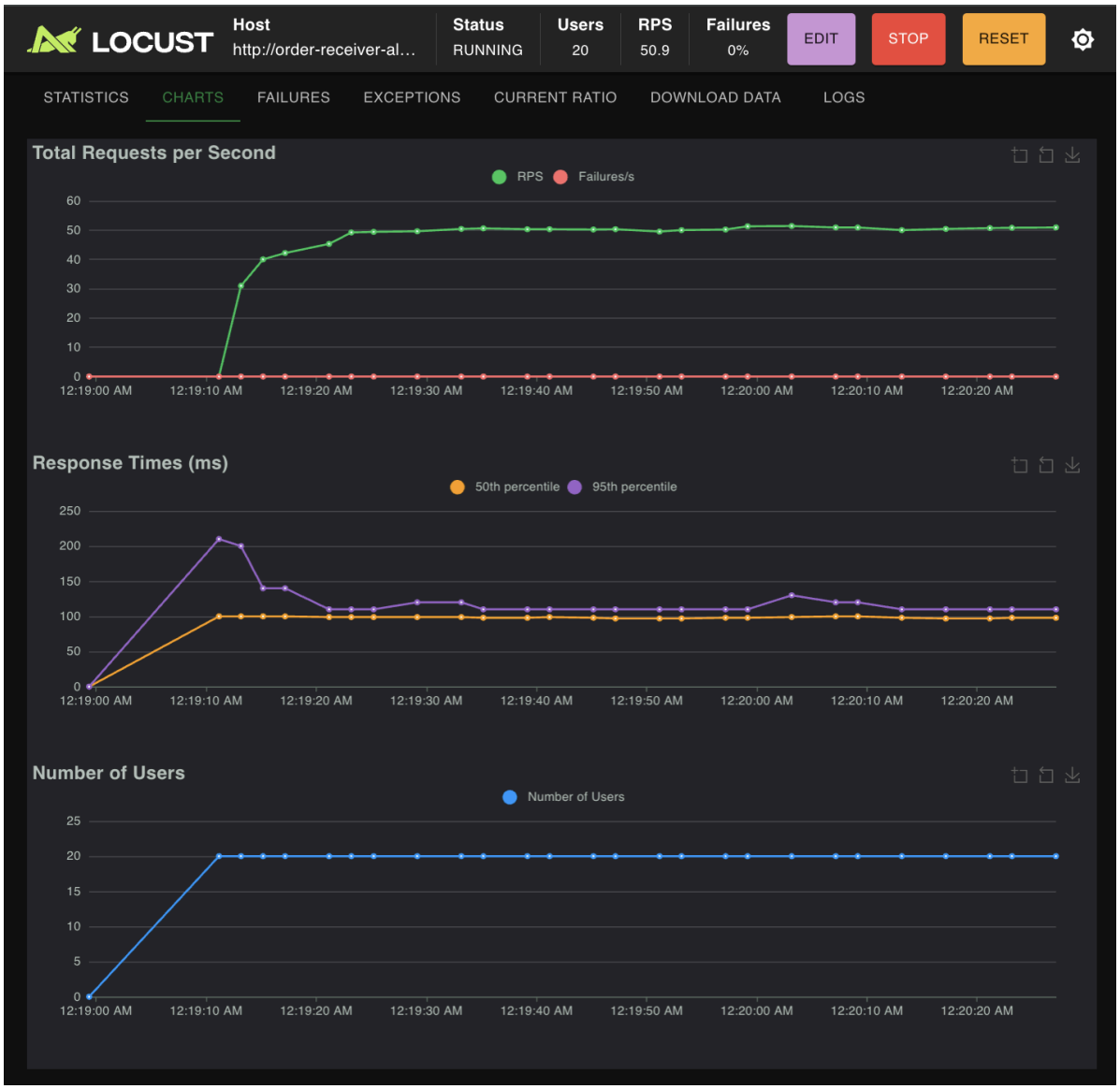
The queue flatlined at the top , it did not drain at all during the test. With 1 worker, it would take over 3 hours to fully process the backlog. This is the hidden cost of async: the customer gets an instant acknowledgment, but their order may not actually be processed for hours. Customer service starts getting calls: *"Where's my order confirmation?"*

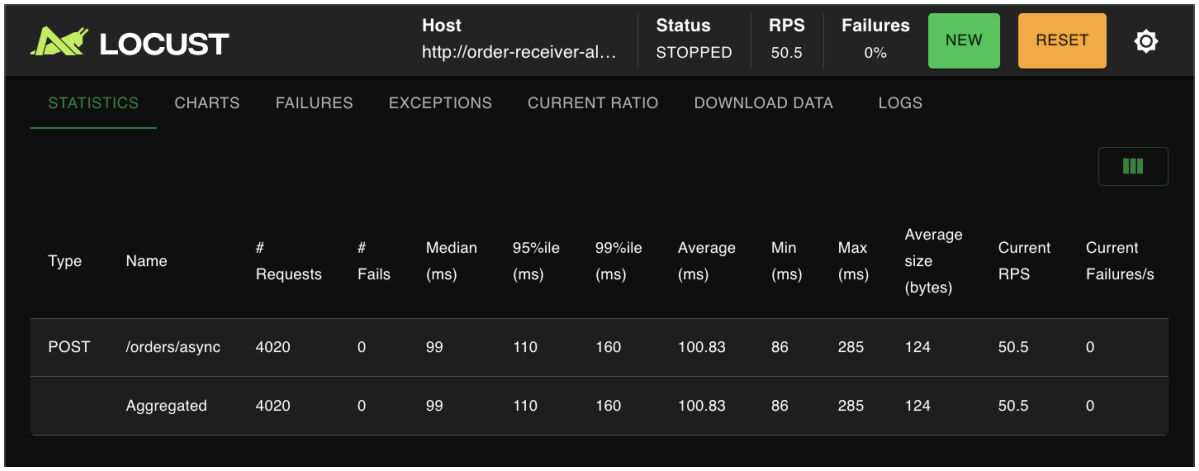
This reveals that accepting orders and processing orders are two separate problems. Fixing one without the other just shifts the bottleneck downstream.

Name: Manasha Sekar

Phase 5

Workers:5





Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders/async	4020	0	99	110	160	100.83	86	285	124	50.5	0
	Aggregated	4020	0	99	110	160	100.83	86	285	124	50.5	0

Name: Manasha Sekar

Cloud Watch

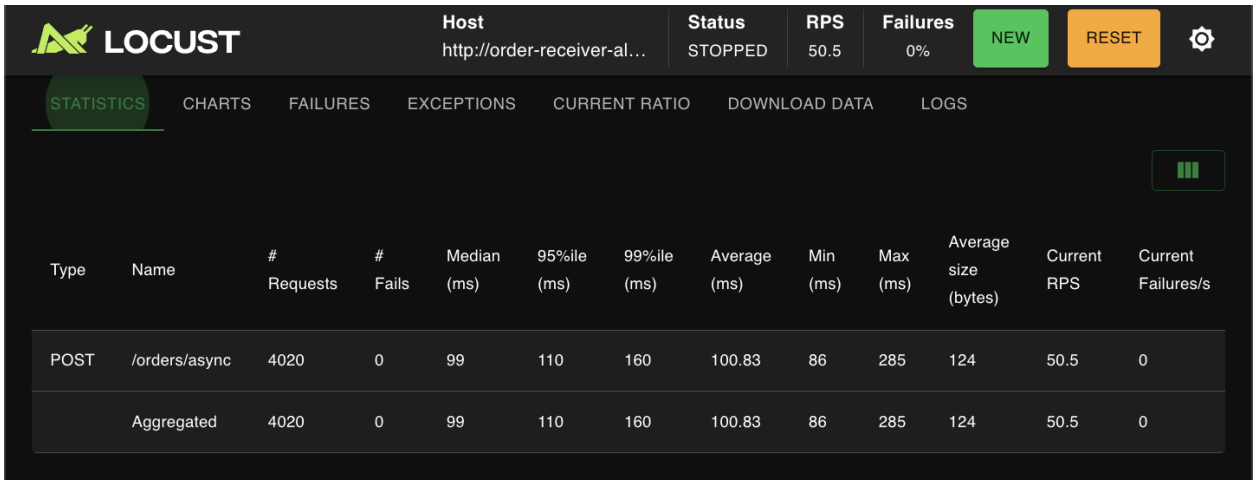


- RPS: 50.5
- Failures: 0%
- Avg Response Time: ~100ms
- Processing rate: 5 ÷ 3 = 1.67 orders/second
- Peak queue depth: ~3,870 messages
- Time to clear backlog: ~38 minutes

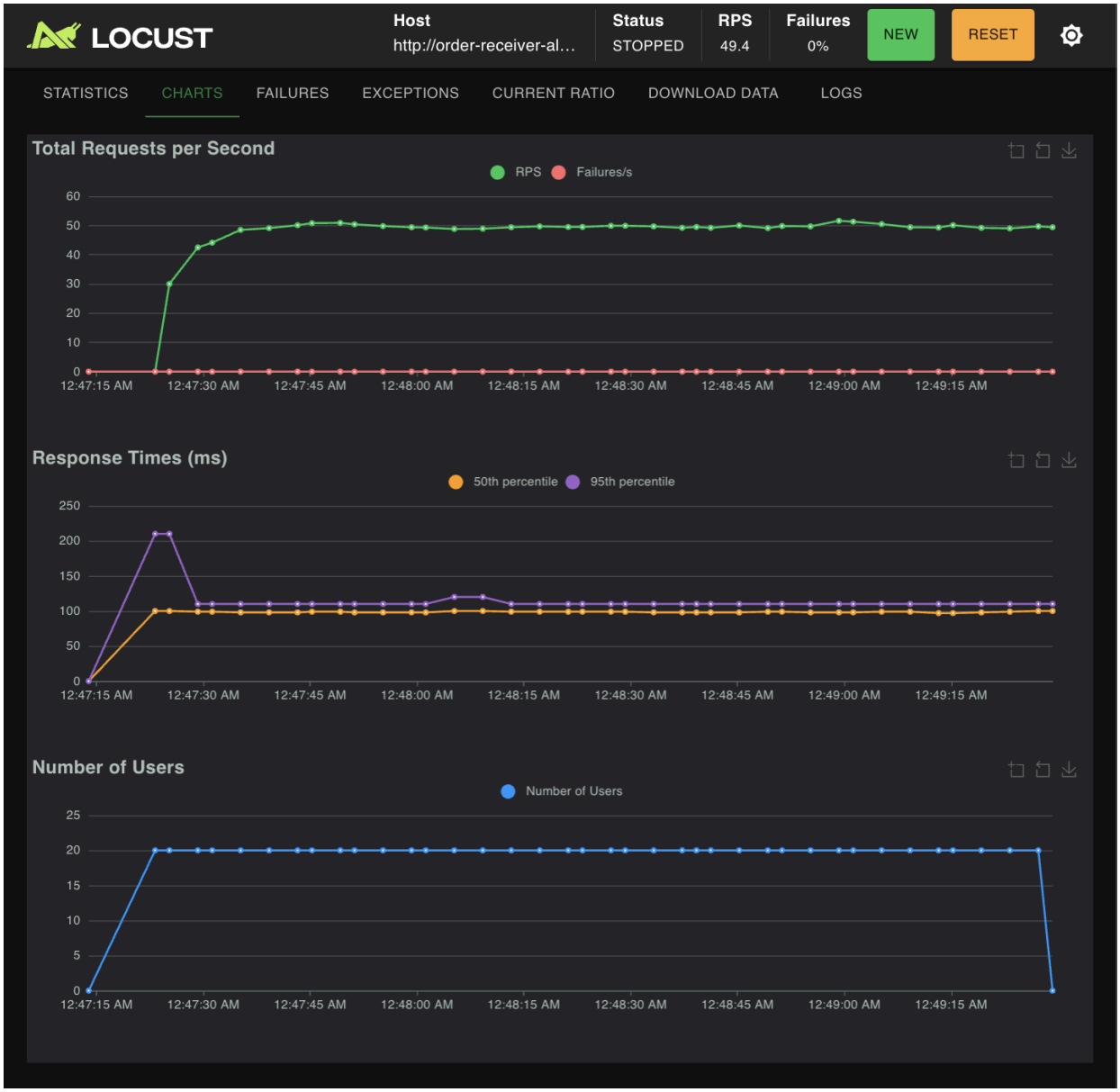
Name: Manasha Sekar

Workers: 20

Ran it for 2 mins.

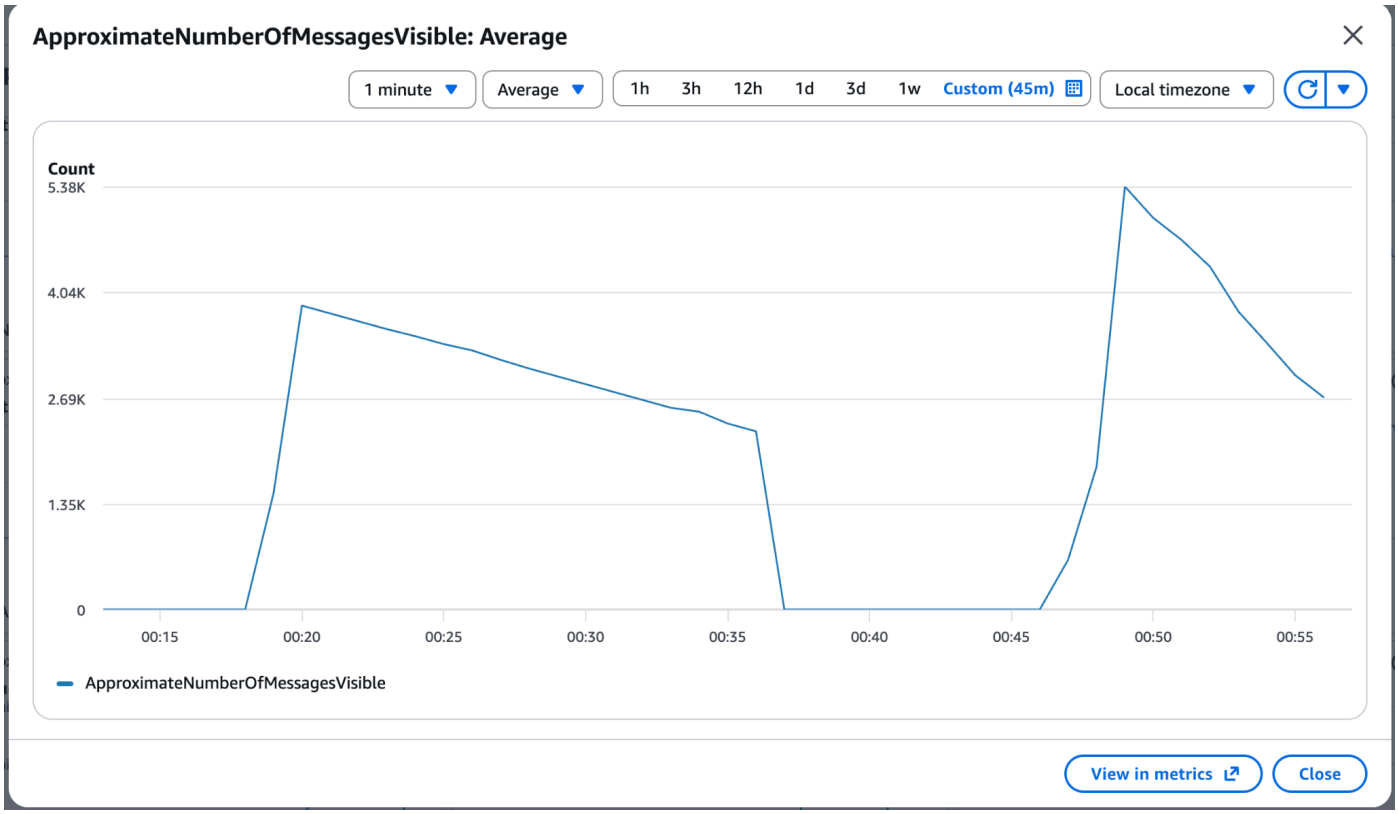


Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders/async	4020	0	99	110	160	100.83	86	285	124	50.5	0
	Aggregated	4020	0	99	110	160	100.83	86	285	124	50.5	0



Name: Manasha Sekar

Cloud Watch:

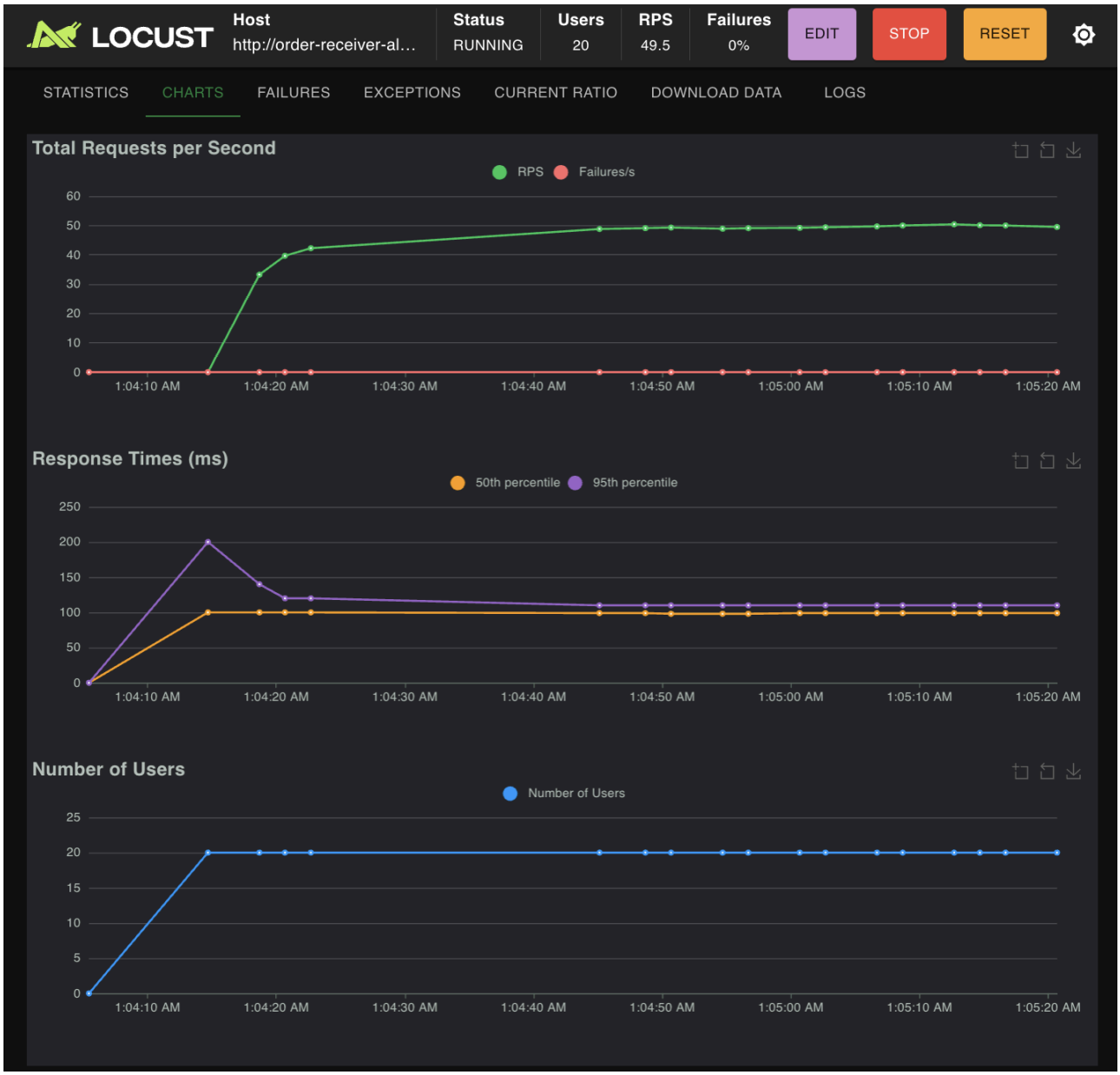


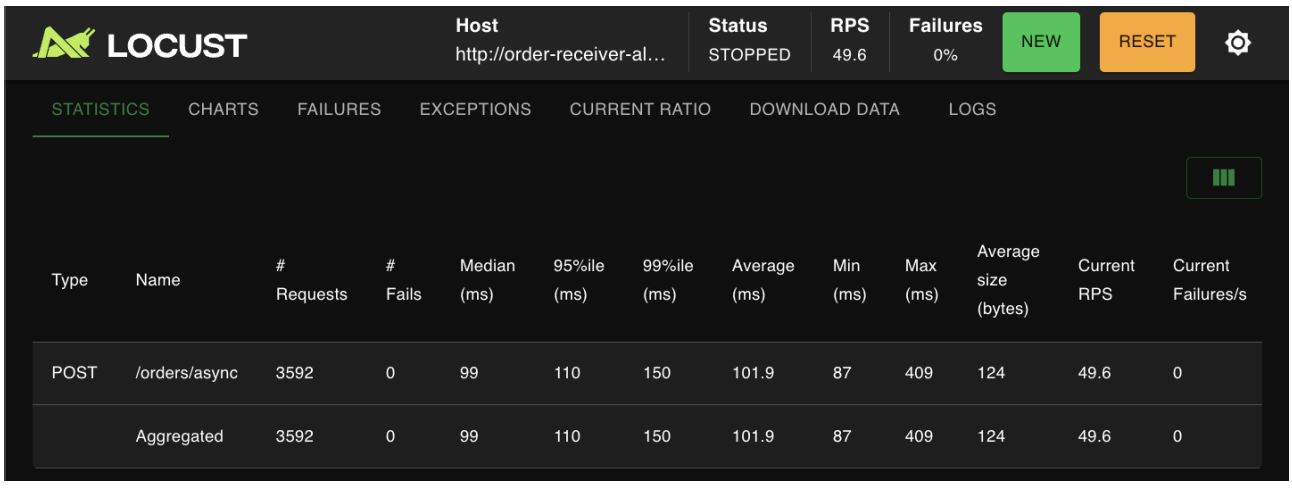
Test Configuration: 20 users, 120 seconds, spawn rate 10/s

- RPS: 50.5
- Failures: 0%
- Avg Response Time: ~100ms
- Processing rate: 20 ÷ 3 = 6.67 orders/second
- Peak queue depth: ~2,690 messages (normalized to 60s)
- Time to clear backlog: ~10 minutes

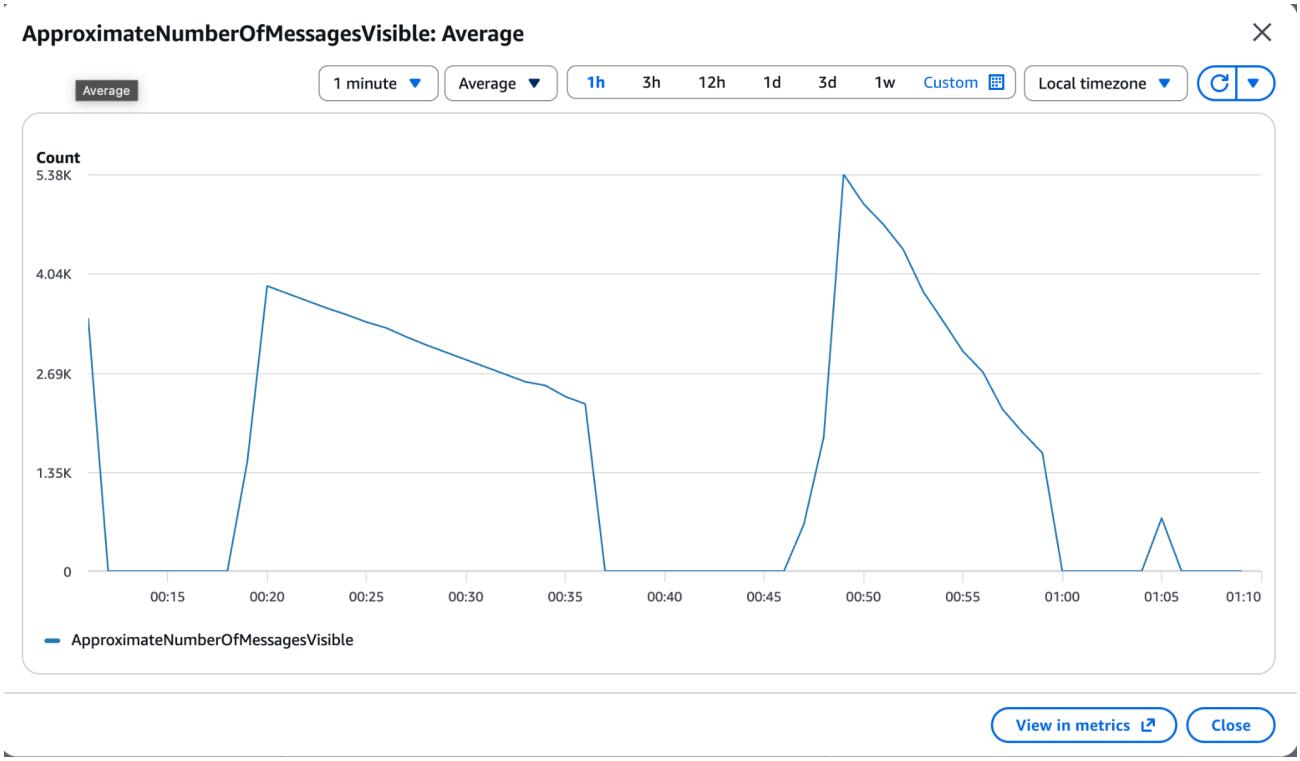
Name: Manasha Sekar

Workers:100





Name: Manasha Sekar



- RPS: 49.6
- Failures: 0%
- Avg Response Time: ~101ms
- Processing rate: 100 ÷ 3 = 33 orders/second
- Peak queue depth: ~900 messages
- Time to clear backlog: ~2 minutes

Summary

Workers	Processing Rate	Peak Queue	Time to Clear
1	0.33/s	~3,870	~192 minutes
5	1.67/s	~3,870	~38 minutes
20	6.67/s	~2,690	~10 minutes
100	33/s	~900	~2 minutes

To fully prevent queue buildup at 50 orders/second, you would need at least **150 workers** (150 ÷ 3 = 50/s). However, a single ECS task with 256 CPU units and 512MB memory would be heavily throttled well before reaching that number, making horizontal scaling (more ECS tasks) or Lambda a more practical solution.

Name: Manasha Sekar

Analysis Questions

1. How many times more orders did your asynchronous approach accept compared to your synchronous approach?

The async endpoint accepted 3,872 orders whereas 26 with the synchronous approach roughly 149x more orders in the same 60-second window. The sync endpoint was bottlenecked by the 3-second payment verification, limiting it to 0.33 orders/second regardless of how many users were sending requests. The async endpoint decoupled acceptance from processing, allowing it to sustain ~50 orders/second with sub-100ms response times throughout the flash sale.

---

2. What causes queue buildup and how do you prevent it?

Queue buildup occurs when the rate of incoming messages exceeds the rate at which workers can process them. In our case, the async endpoint accepted ~50 orders/second while a single worker could only process 0.33 orders/second — a gap of ~49.67 messages/second. The queue grew to ~3,870 messages in just 60 seconds and would have taken over 3 hours to drain with 1 worker.

The solution is to match processing capacity to the incoming rate. This can be done by scaling the number of worker goroutines (as shown in Phase 5), scaling ECS tasks horizontally, or using auto-scaling based on the CloudWatch metric. At 50 orders/second with a 3-second processing time, at least 150 concurrent workers are needed to prevent buildup entirely.

---

3. When would you choose sync vs async in production?

Choose **synchronous** processing when the result is needed immediately by the caller. Sync is simpler to reason about, easier to debug, and provides immediate feedback on success or failure. It works well when the operation is fast and load is predictable.

Choose **asynchronous** processing when the operation is slow, the result is not needed immediately, or the system must absorb unpredictable bursts of traffic. Order processing, email sending, report generation, and video transcoding are all good candidates.
The trade-off is added complexity: you need a queue, background workers, and a way to notify users when processing completes. You also lose the immediate guarantee that the operation succeeded, a 202 Accepted only means the order was received, not that payment was actually verified.

Name: Manasha Sekar

PART III

```
(base) manashasekar@Manashas-MacBook-Air BSDS % for i in {1..10}; do
  curl -X POST http://order-receiver-alb-272010506.us-west-2.elb.amazonaws.com/orders/async \
    -H "Content-Type: application/json" \
    -d "{\"customer_id\": $i, \"items\": [{\"name\": \"shirt\", \"quantity\": 1, \"price\": 29.99}]}"
  echo ""
done
{"message":"order received, processing in background","order_id":"af87a5f5-160a-4d15-8cfe-f42a0ae478f7","status":"pending"}

{"message":"order received, processing in background","order_id":"a0c2a5d0-1c3b-4624-9db7-242314c20436","status":"pending"}

{"message":"order received, processing in background","order_id":"ed94f021-7393-4b52-9017-12db81abcc5e","status":"pending"}

{"message":"order received, processing in background","order_id":"16f1df82-e4fe-4c67-b9c3-345180b4694a","status":"pending"}

{"message":"order received, processing in background","order_id":"3aacc863-8350-4833-ab84-18bd7a72daa9","status":"pending"}

{"message":"order received, processing in background","order_id":"ad4de481-88f0-4111-ad2a-015b632cda0c","status":"pending"}

{"message":"order received, processing in background","order_id":"0075913a-c00c-4961-addd-140278bdd5b6","status":"pending"}

{"message":"order received, processing in background","order_id":"10f2a46e-2f6a-4ade-a81a-76cc58ba5d65","status":"pending"}

{"message":"order received, processing in background","order_id":"1f305975-4097-4c90-bf88-04a30cac4671","status":"pending"}

{"message":"order received, processing in background","order_id":"10109efa-559d-4f13-94b6-2bd5cbd2d3a5","status":"pending"}
```

```
(base) manashasekar@Manashas-MacBook-Air BSDS % aws logs tail /aws/lambda/order-lambda --follow --region us-west-2
2026-03-16T06:08:38.693000+00:00 2026/03/16/[$LATEST]4e27cdfaa85c4a25bd40c1a3f01d923e START RequestId: 29c3f02f-f880-4a44-ba8a-505112e99f93 Version: $LATEST
2026-03-16T06:08:38.695000+00:00 2026/03/16/[$LATEST]43a20fb07cc24d6f9e35588ad65907e9 START RequestId: eefcdeaa-02f5-4904-8f15-3c393e9b36ac Version: $LATEST
2026-03-16T06:08:38.697000+00:00 2026/03/16/[$LATEST]304c616572354aedbf77b73950c71ce7 START RequestId: c22fe879-27c2-45c0-999b-49cc7c2dc926 Version: $LATEST
2026-03-16T06:08:38.698000+00:00 2026/03/16/[$LATEST]a78f878b78594c499f906693da173f43 START RequestId: b1643665-8e44-4345-a460-b4930c9f98f7 Version: $LATEST
2026-03-16T06:08:38.700000+00:00 2026/03/16/[$LATEST]4e27cdfaa85c4a25bd40c1a3f01d923e 2026/03/16 06:08:38 processing order 3aacc863-8350-4833-ab84-18bd7a72daa9 for customer 5
2026-03-16T06:08:38.700000+00:00 2026/03/16/[$LATEST]f81fa4f414d54ff5bdcc0b860e5da740 START RequestId: 477d2e5c-89a2-426d-bd58-0072c27adb08 Version: $LATEST
2026-03-16T06:08:38.701000+00:00 2026/03/16/[$LATEST]43a20fb07cc24d6f9e35588ad65907e9 2026/03/16 06:08:38 processing order a0c2a5d0-1c3b-4624-9db7-242314c20436 for customer 2
2026-03-16T06:08:38.703000+00:00 2026/03/16/[$LATEST]a78f878b78594c499f906693da173f43 2026/03/16 06:08:38 processing order 16f1df82-e4fe-4c67-b9c3-345180b4694a for customer 4
2026-03-16T06:08:38.703000+00:00 2026/03/16/[$LATEST]304c616572354aedbf77b73950c71ce7 2026/03/16 06:08:38 processing order 10f2a46e-2f6a-4ade-a81a-76cc58ba5d65 for customer 8
2026-03-16T06:08:38.704000+00:00 2026/03/16/[$LATEST]c8995b67b4414c19830ba6f1444fbb2d START RequestId: ee8d54c0-a499-4be4-b00a-42c5a8511f12 Version: $LATEST
2026-03-16T06:08:38.705000+00:00 2026/03/16/[$LATEST]f81fa4f414d54ff5bdcc0b860e5da740 2026/03/16 06:08:38 processing order af87a5f5-160a-4d15-8cfe-f42a0ae478f7 for customer 1
2026-03-16T06:08:38.711000+00:00 2026/03/16/[$LATEST]c8995b67b4414c19830ba6f1444fbb2d 2026/03/16 06:08:38 processing order 0075913a-c00c-4961-addd-140278bdd5b6 for customer 7
2026-03-16T06:08:38.810000+00:00 2026/03/16/[$LATEST]8eaa659544a94eb0b7b0d101dcc2acad START RequestId: 81ef48d7-2ad7-49c5-bc98-00a2bc39b3dd Version: $LATEST
2026-03-16T06:08:38.815000+00:00 2026/03/16/[$LATEST]8eaa659544a94eb0b7b0d101dcc2acad 2026/03/16 06:08:38 processing order ad4de481-88f0-4111-ad2a-015b632cda0c for customer 6
2026-03-16T06:08:38.968000+00:00 2026/03/16/[$LATEST]145f5a3f1c3c4bd8b812319abeee56bb START RequestId: 31fa0520-4414-4e91-8387-12e672170f27 Version: $LATEST
2026-03-16T06:08:38.974000+00:00 2026/03/16/[$LATEST]145f5a3f1c3c4bd8b812319abeee56bb 2026/03/16 06:08:38 processing order 1f305975-4097-4c90-bf88-04a30cac4671 for customer 9
2026-03-16T06:08:39.155000+00:00 2026/03/16/[$LATEST]cf0fba8446ca4853b43412ea3c2b584d START RequestId: e3a21687-79ab-4956-862d-c6fc2b3ea4b5 Version: $LATEST
2026-03-16T06:08:39.164000+00:00 2026/03/16/[$LATEST]cf0fba8446ca4853b43412ea3c2b584d 2026/03/16 06:08:39 processing order 10109efa-559d-4f13-94b6-2bd5cbd2d3a5 for customer 10
2026-03-16T06:08:41.702000+00:00 2026/03/16/[$LATEST]4e27cdfaa85c4a25bd40c1a3f01d923e 2026/03/16 06:08:41 completed order 3aacc863-8350-4833-ab84-18bd7a72daa9
2026-03-16T06:08:41.703000+00:00 2026/03/16/[$LATEST]304c616572354aedbf77b73950c71ce7 2026/03/16 06:08:41 completed order 10f2a46e-2f6a-4ade-a81a-76cc58ba5d65
2026-03-16T06:08:41.704000+00:00 2026/03/16/[$LATEST]4e27cdfaa85c4a25bd40c1a3f01d923e END RequestId: 29c3f02f-f880-4a44-ba8a-505112e99f93
2026-03-16T06:08:41.704000+00:00 2026/03/16/[$LATEST]4e27cdfaa85c4a25bd40c1a3f01d923e REPORT RequestId: 29c3f02f-f880-4a44-ba8a-505112e99f93        Duration: 3010.20 ms    Billed Durati
on: 3708 ms  Memory Size: 512 MB     Max Memory Used: 20 MB     Init Duration: 697.47 ms
2026-03-16T06:08:41.705000+00:00 2026/03/16/[$LATEST]304c616572354aedbf77b73950c71ce7 END RequestId: c22fe879-27c2-45c0-999b-49cc7c2dc926
2026-03-16T06:08:41.705000+00:00 2026/03/16/[$LATEST]304c616572354aedbf77b73950c71ce7 REPORT RequestId: c22fe879-27c2-45c0-999b-49cc7c2dc926        Duration: 3007.13 ms    Billed Durati
on: 3144 ms  Memory Size: 512 MB     Max Memory Used: 20 MB     Init Duration: 136.24 ms
2026-03-16T06:08:41.705000+00:00 2026/03/16/[$LATEST]f81fa4f414d54ff5bdcc0b860e5da740 2026/03/16 06:08:41 completed order af87a5f5-160a-4d15-8cfe-f42a0ae478f7
2026-03-16T06:08:41.707000+00:00 2026/03/16/[$LATEST]f81fa4f414d54ff5bdcc0b860e5da740 END RequestId: 477d2e5c-89a2-426d-bd58-0072c27adb08
2026-03-16T06:08:41.707000+00:00 2026/03/16/[$LATEST]f81fa4f414d54ff5bdcc0b860e5da740 REPORT RequestId: 477d2e5c-89a2-426d-bd58-0072c27adb08        Duration: 3007.21 ms    Billed Durati
on: 4292 ms  Memory Size: 512 MB     Max Memory Used: 20 MB     Init Duration: 1284.70 ms
2026-03-16T06:08:41.816000+00:00 2026/03/16/[$LATEST]8eaa659544a94eb0b7b0d101dcc2acad 2026/03/16 06:08:41 completed order ad4de481-88f0-4111-ad2a-015b632cda0c
2026-03-16T06:08:41.818000+00:00 2026/03/16/[$LATEST]8eaa659544a94eb0b7b0d101dcc2acad END RequestId: 81ef48d7-2ad7-49c5-bc98-00a2bc39b3dd
2026-03-16T06:08:41.818000+00:00 2026/03/16/[$LATEST]8eaa659544a94eb0b7b0d101dcc2acad REPORT RequestId: 81ef48d7-2ad7-49c5-bc98-00a2bc39b3dd        Duration: 3007.60 ms    Billed Durati
on: 3481 ms  Memory Size: 512 MB     Max Memory Used: 20 MB     Init Duration: 472.88 ms
2026-03-16T06:08:41.976000+00:00 2026/03/16/[$LATEST]145f5a3f1c3c4bd8b812319abeee56bb 2026/03/16 06:08:41 completed order 1f305975-4097-4c90-bf88-04a30cac4671
2026-03-16T06:08:41.978000+00:00 2026/03/16/[$LATEST]145f5a3f1c3c4bd8b812319abeee56bb END RequestId: 31fa0520-4414-4e91-8387-12e672170f27
2026-03-16T06:08:41.978000+00:00 2026/03/16/[$LATEST]145f5a3f1c3c4bd8b812319abeee56bb REPORT RequestId: 31fa0520-4414-4e91-8387-12e672170f27        Duration: 3009.61 ms    Billed Durati
on: 3087 ms  Memory Size: 512 MB     Max Memory Used: 20 MB     Init Duration: 76.72 ms
2026-03-16T06:08:42.166000+00:00 2026/03/16/[$LATEST]cf0fba8446ca4853b43412ea3c2b584d 2026/03/16 06:08:42 completed order 10109efa-559d-4f13-94b6-2bd5cbd2d3a5
2026-03-16T06:08:42.168000+00:00 2026/03/16/[$LATEST]cf0fba8446ca4853b43412ea3c2b584d END RequestId: e3a21687-79ab-4956-862d-c6fc2b3ea4b5
2026-03-16T06:08:42.168000+00:00 2026/03/16/[$LATEST]cf0fba8446ca4853b43412ea3c2b584d REPORT RequestId: e3a21687-79ab-4956-862d-c6fc2b3ea4b5        Duration: 3012.70 ms    Billed Durati
on: 3089 ms  Memory Size: 512 MB     Max Memory Used: 20 MB     Init Duration: 76.01 ms
```

```
(base) manashasekar@Manashas-MacBook-Air BSDS % aws logs tail /aws/lambda/order-lambda --follow --region us-west-2
(base) manashasekar@Manashas-MacBook-Air BSDS % curl -X POST http://order-receiver-alb-272010506.us-west-2.elb.amazonaws.com/orders/async \
  -H "Content-Type: application/json" \
  -d '{"customer_id": 123, "items": [{"name": "shirt", "quantity": 1, "price": 29.99}]}'
{"message":"order received, processing in background","order_id":"5c4fa7c2-3d9d-43d0-80e4-49eb285bcf0d","status":"pending"}
```

Name: Manasha Sekar



1. Deployed Lambda Function

A Lambda function was deployed using the provided.al2 runtime with 512MB memory, subscribed directly to the existing SNS topic order-processing-events. Orders flow from the API -> SNS -> Lambda with no SQS queue in between. The same 3-second payment simulation runs inside the Lambda handler.

2. Cold Start Observations

10 orders were sent simultaneously to trigger concurrent Lambda invocations. Since all 10 fired at the same time, Lambda spun up 10 fresh instances in parallel, every invocation was a cold start.

Cold start results (10 concurrent invocations):

Request	Duration	Init Duration
customer 5	3010ms	697ms
customer 8	3007ms	136ms

customer 1	3007ms	1284ms
customer 2	3009ms	1225ms
customer 3	3010ms	1083ms
customer 4	3009ms	747ms
customer 6	3007ms	472ms
customer 7	3008ms	325ms
customer 9	3009ms	76ms
customer 10	3012ms	76ms

- Cold start overhead ranged from **76ms to 1284ms**
- High variance because all 10 instances initialized simultaneously competing for resources
- Memory used was only **20MB out of 512MB**.

Warm start (after ~8 minutes idle):

REPORT Duration: 3003.66ms, Max Used: 20MB

No Init Duration, Lambda reused an existing instance. The warm start added **0ms overhead** on top of the 3 second processing time.

Cold starts occur: on first invocation, after ~5+ minutes idle, and when concurrent requests exceed the number of warm instances.

3. Cost Calculation

Current ECS cost:

- 2 ECS tasks × $8.50/month = **$17/month** (always running)

Name: Manasha Sekar

Lambda cost (AWS free tier):

- Free tier: 1M requests + 400K GB-seconds/month
- Per order: 3 seconds × 0.5GB = 1.5 GB-seconds
- Free orders per month: 400,000 ÷ 1.5 = **~267,000 orders FREE**

Monthly Volume	Lambda Cost	ECS Cost
10,000 orders	$0 (free tier)	$17
100,000 orders	$0 (free tier)	$17
267,000 orders	$0 (free tier)	$17
1,000,000 orders	~$5.50	$17

Break-even: Lambda costs more than ECS only beyond ~1.7M orders/month.

4. Switch Recommendation

For our startup at current order volumes, switching to Lambda makes sense. Lambda eliminated all operational overhead, no SQS queue to monitor, no worker goroutine tuning, no ECS health checks. The cost advantage is compelling: at under 267,000 orders per month Lambda is completely free. Cold starts ranged from 76ms to 1284ms but averaged around 400ms on top of a 3 second payment processing time. The main trade-off is losing SQS guarantees: SNS retries Lambda only twice before discarding the message, whereas SQS provides configurable retries, dead-letter queues, and visibility timeouts. For a startup under 267K orders/month where operational simplicity matters more than message durability guarantees, Lambda is the right choice. As the business scales past 1M+ orders/month or requires stricter delivery guarantees, revisiting the SQS + ECS architecture would be warranted.

ANANYAA MYLSAMY
002022427

HW7: WHEN YOUR STARTUP'S FLASH SALE ALMOST FAILED

PART II - ASYNC ORDER PROCESSING WITH SNS AND SQS

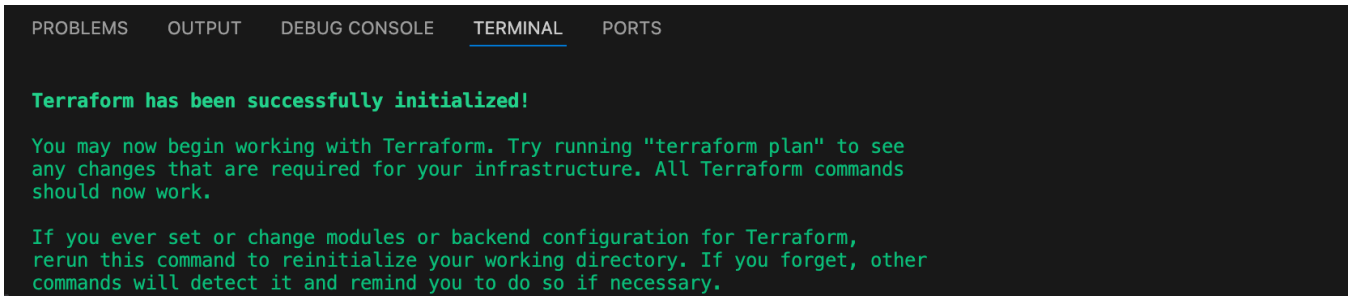
An e-commerce order processing service that breaks under flash sale load using synchronous processing, then gets fixed with an event-driven async architecture using SNS and SQS.

A Go service with two endpoints: /orders/sync which blocks for 3 seconds simulating payment verification using a buffered channel semaphore, and /orders/async which publishes to SNS and returns 202 immediately. A separate ECS processor service polls SQS and processes orders with the same 3-second delay.

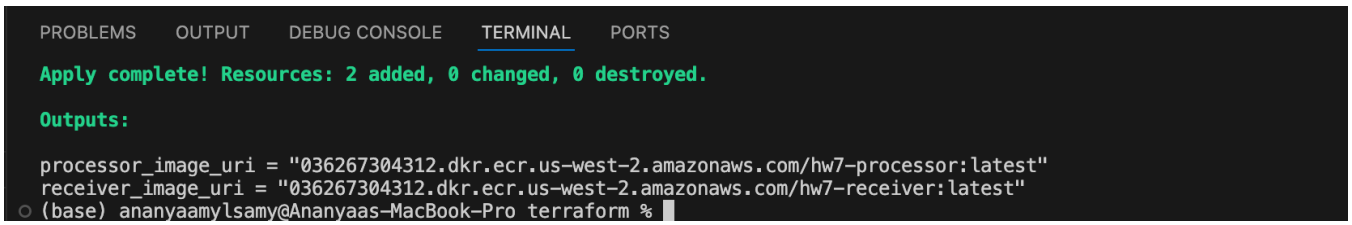
Infrastructure

- **VPC**: 10.0.0.0/16, public subnets for ALB, private subnets for ECS
- **ECS Fargate**: 256 CPU, 512MB - Order Receiver + Order Processor
- **SNS/SQS**: order-processing-events -> order-processing-queue

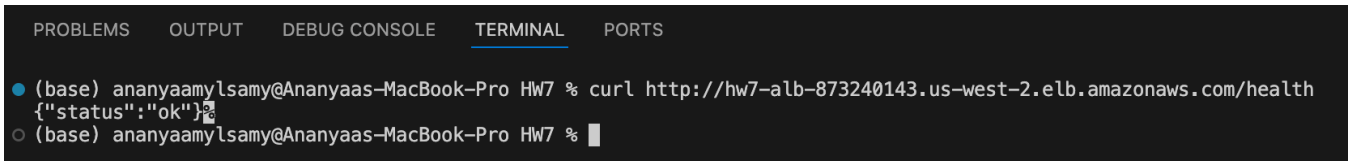
Terraform init



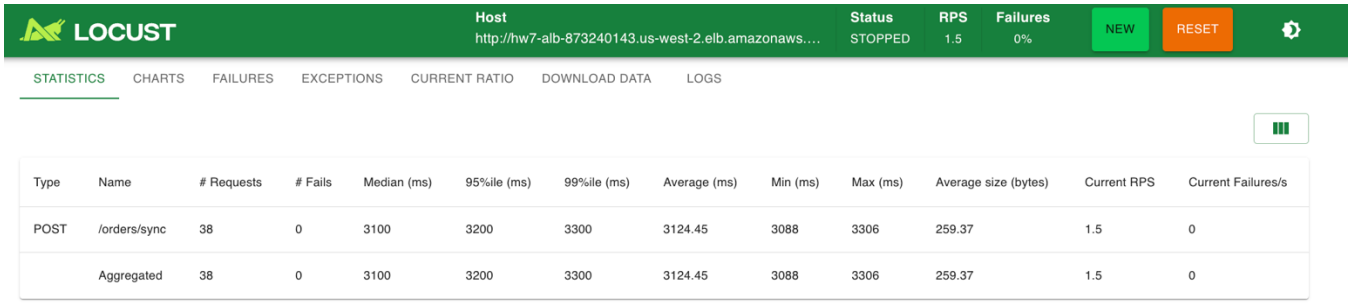
Terraform apply



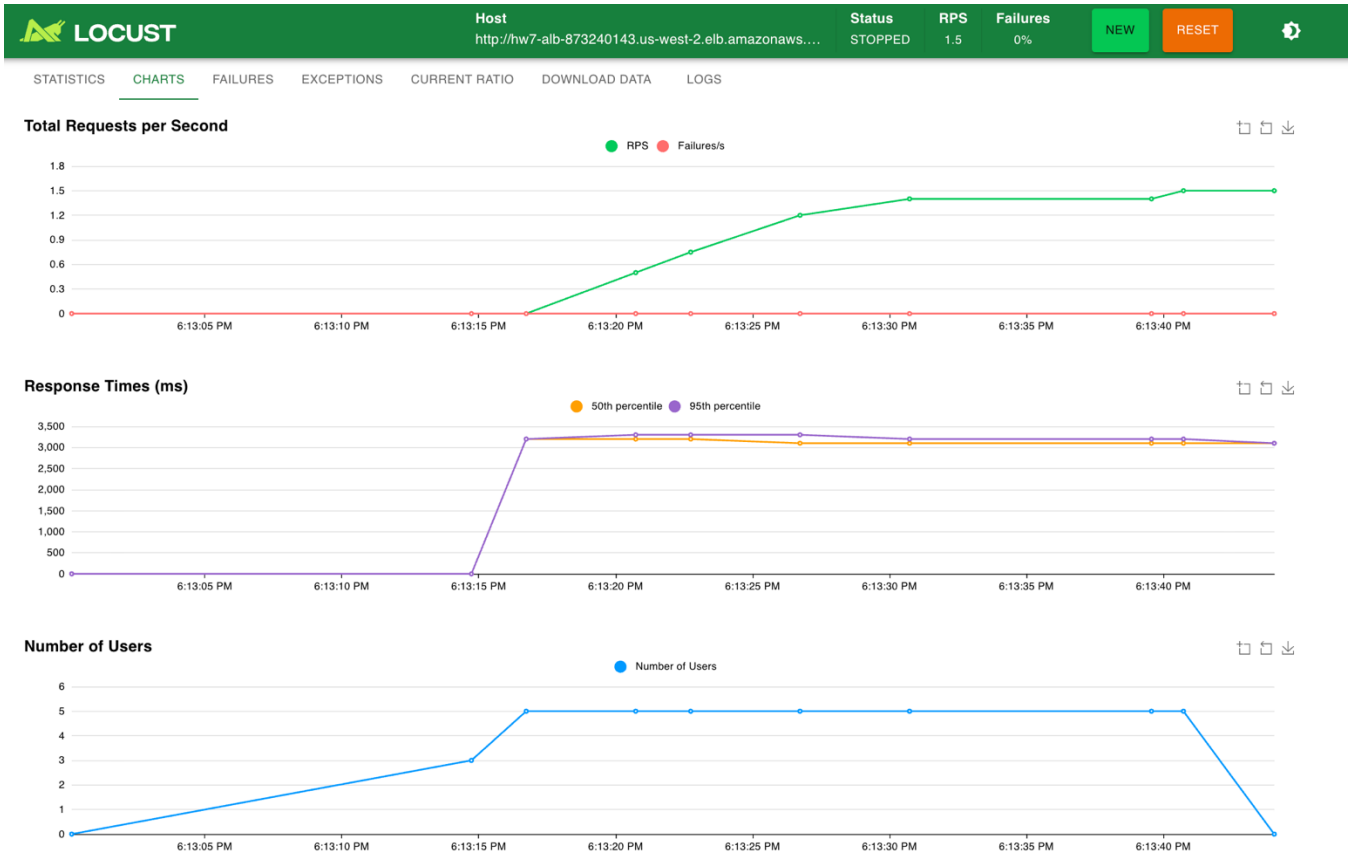
API health check



Phase 1 - Normal Operations: 5 users, 30 seconds

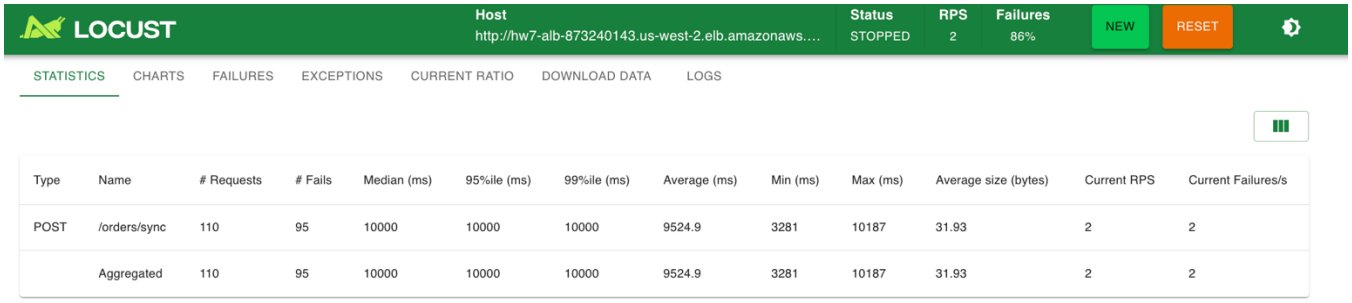


Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders/sync	38	0	3100	3200	3300	3124.45	3088	3306	259.37	1.5	0
	Aggregated	38	0	3100	3200	3300	3124.45	3088	3306	259.37	1.5	0

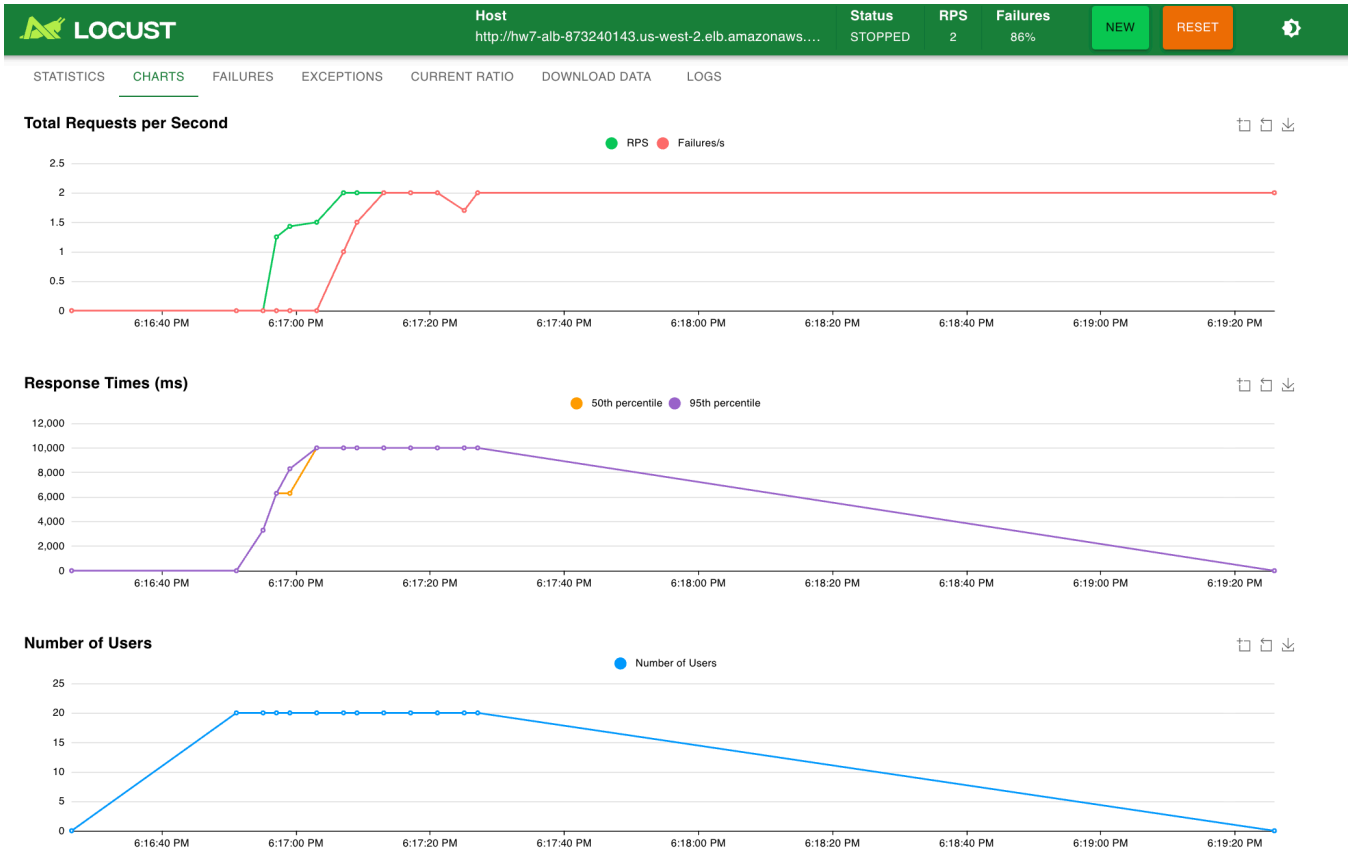


Metric	Value
Requests	38
Failures	0%
RPS	1.5
Median Response	3100ms
95th Percentile	3200ms
99th Percentile	3300ms

Phase 1 - Flash Sale: 20 users, 60 seconds



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders/sync	110	95	10000	10000	10000	9524.9	3281	10187	31.93	2	2
	Aggregated	110	95	10000	10000	10000	9524.9	3281	10187	31.93	2	2



Metric	**Value**
Requests	110
Failures	86% (95 failed)
RPS	2
Median Response	10000ms
95th Percentile	10000ms
99th Percentile	10000ms

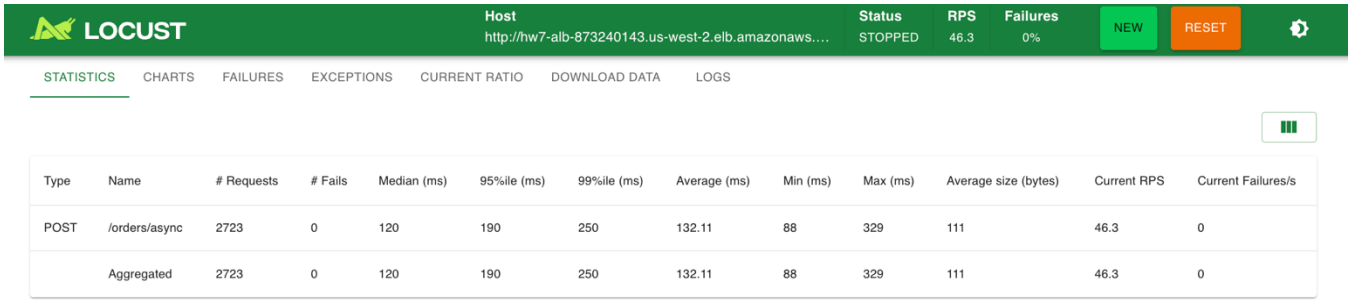
ANANYAA MYLSAMY
002022427

Phase 2: Bottleneck Analysis

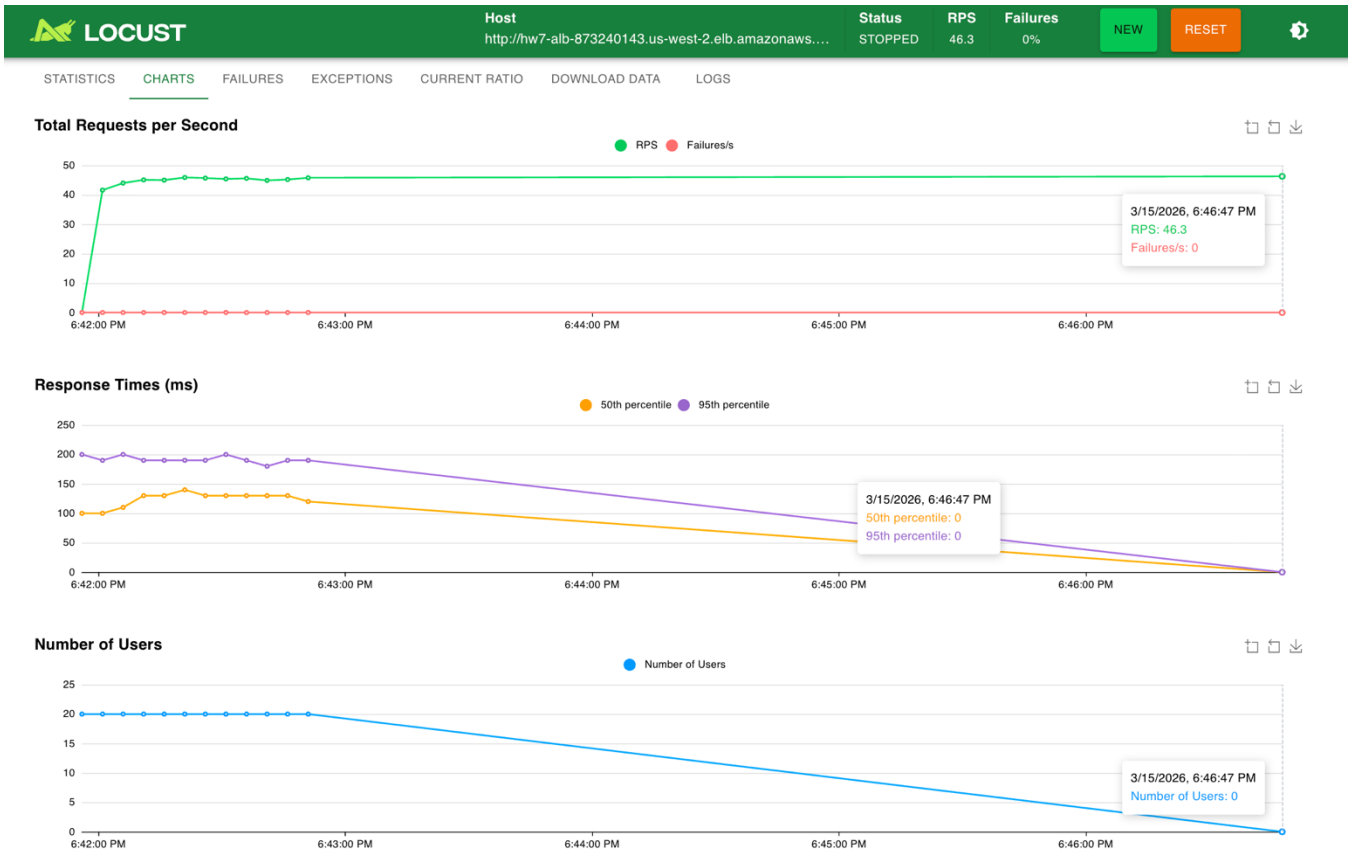
Metric	Value
Payment processor speed	1 order / 3s = 0.33 orders/s
Max throughput	0.33 orders/s
Flash sale demand	20 orders/s
Orders lost per second	19.67 orders/s

Since the payment processor speed is fixed at 0.33 orders/second, the only thing we can change is how we handle the requests. Instead of making customers wait for payment verification, we can acknowledge orders immediately and process payments asynchronously in the background using a queue.

Phase 3: Async Solution



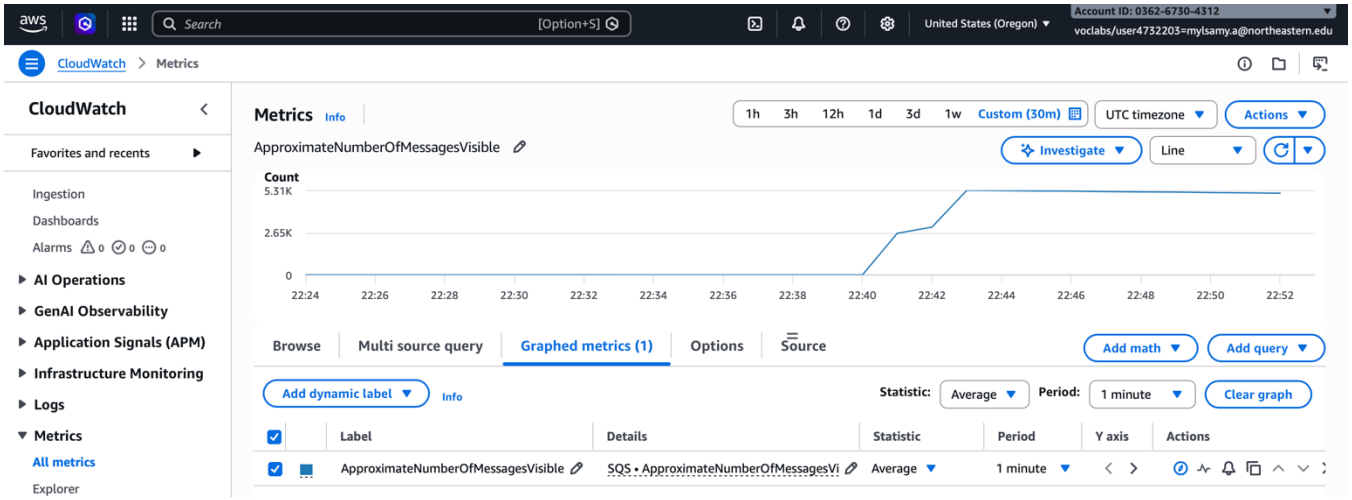
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders/async	2723	0	120	190	250	132.11	88	329	111	46.3	0
	Aggregated	2723	0	120	190	250	132.11	88	329	111	46.3	0



Metric	Value
Requests	2723
Failures	0%
RPS	46.3
Median Response	120ms
95th Percentile	190ms
99th Percentile	250ms

The async endpoint accepted 23x more requests per second compared to sync (46.3 vs 2 RPS) with 0% failures.

Phase 4: Queue Buildup

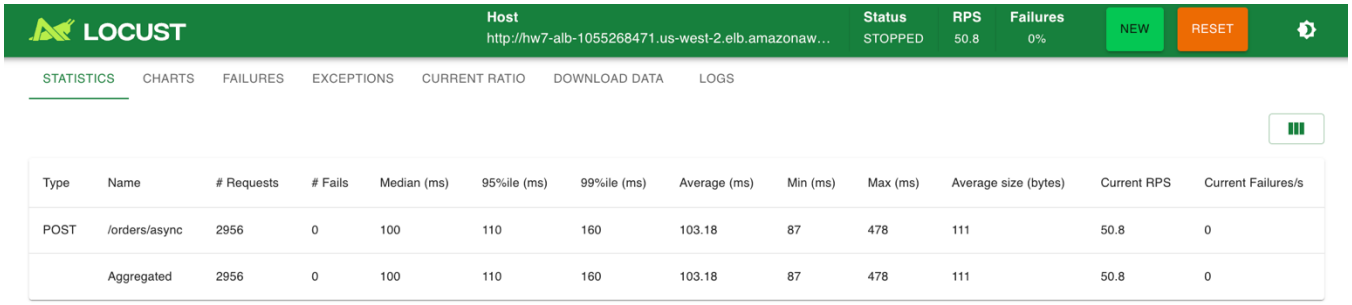


Metric	Value
Order acceptance rate	~46/s
Single worker processing rate	0.33/s
Queue growth rate	~45.7 msg/s
Peak queue depth	~5,310 messages
Time to drain (1 worker)	~267 minutes

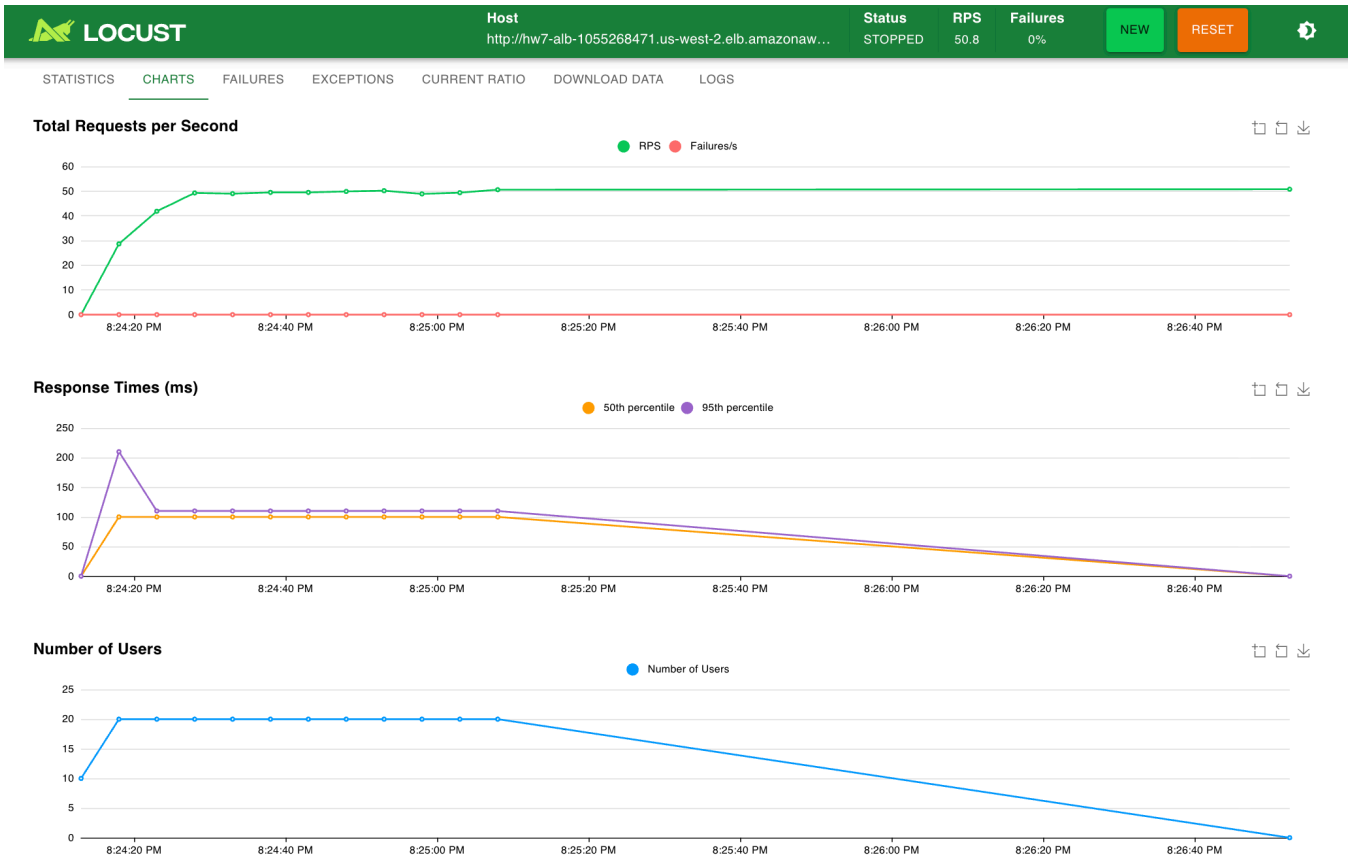
With a single worker, the queue peaked at 5,310 messages and would take approximately 4.5 hours to fully drain - the hidden cost of async processing.

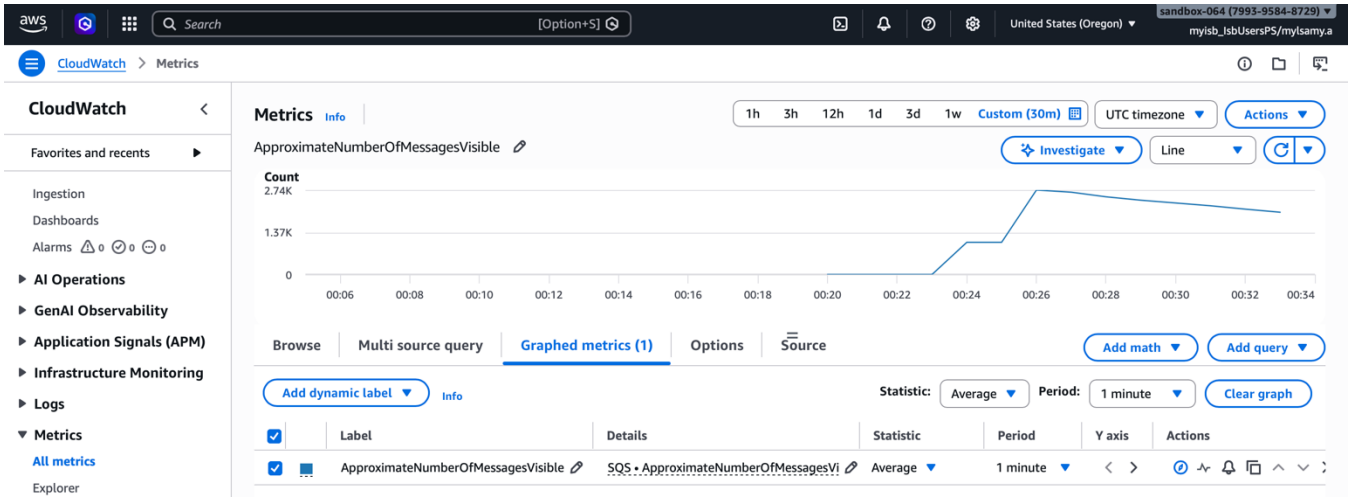
Phase 5: Worker Scaling

5 Workers



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders/async	2956	0	100	110	160	103.18	87	478	111	50.8	0
	Aggregated	2956	0	100	110	160	103.18	87	478	111	50.8	0



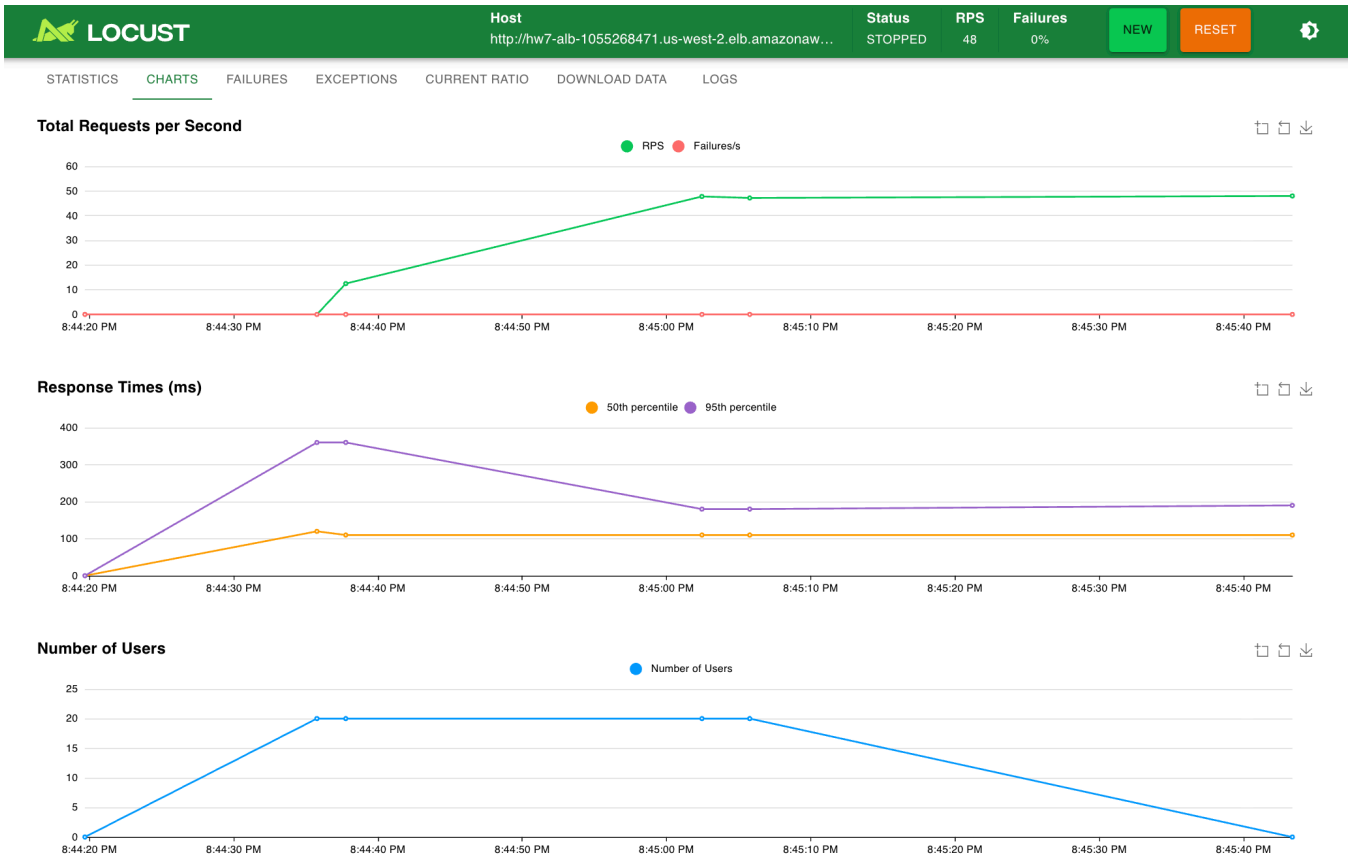


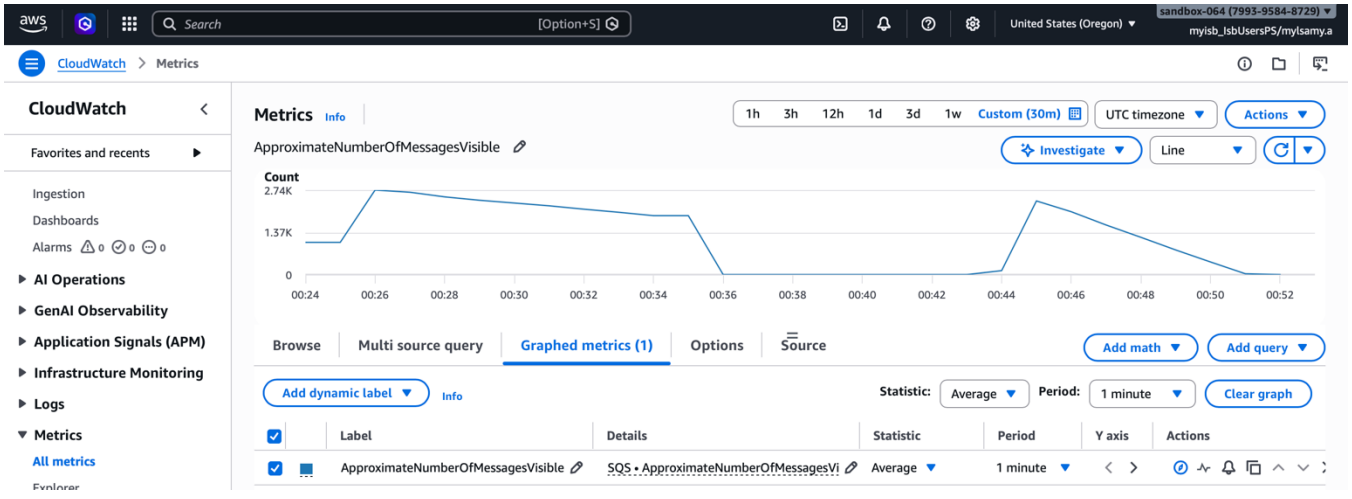
20 Workers

LOCUST | Host: http://hw7-alb-1055268471.us-west-2.elb.amazonaw... | Status: STOPPED | RPS: 48 | Failures: 0% | NEW | RESET

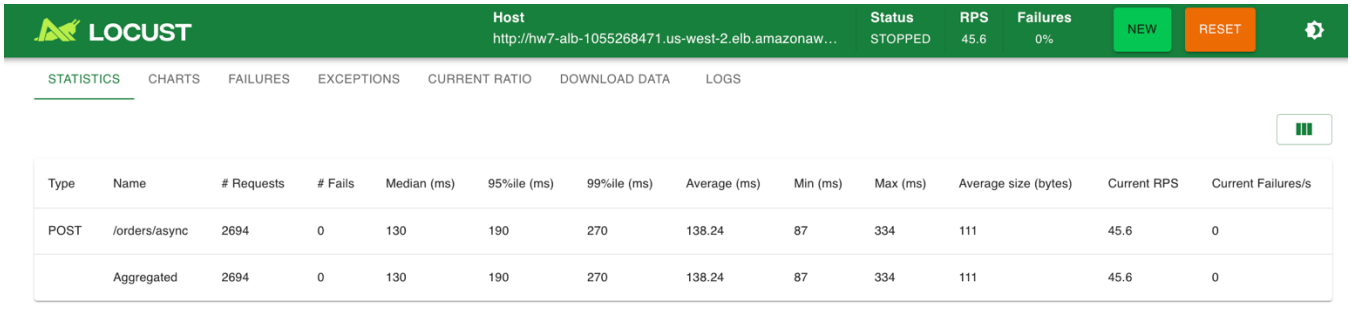
STATISTICS | CHARTS | FAILURES | EXCEPTIONS | CURRENT RATIO | DOWNLOAD DATA | LOGS

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders/async	2809	0	100	180	210	120.95	87	466	111	48	0
	Aggregated	2809	0	100	180	210	120.95	87	466	111	48	0

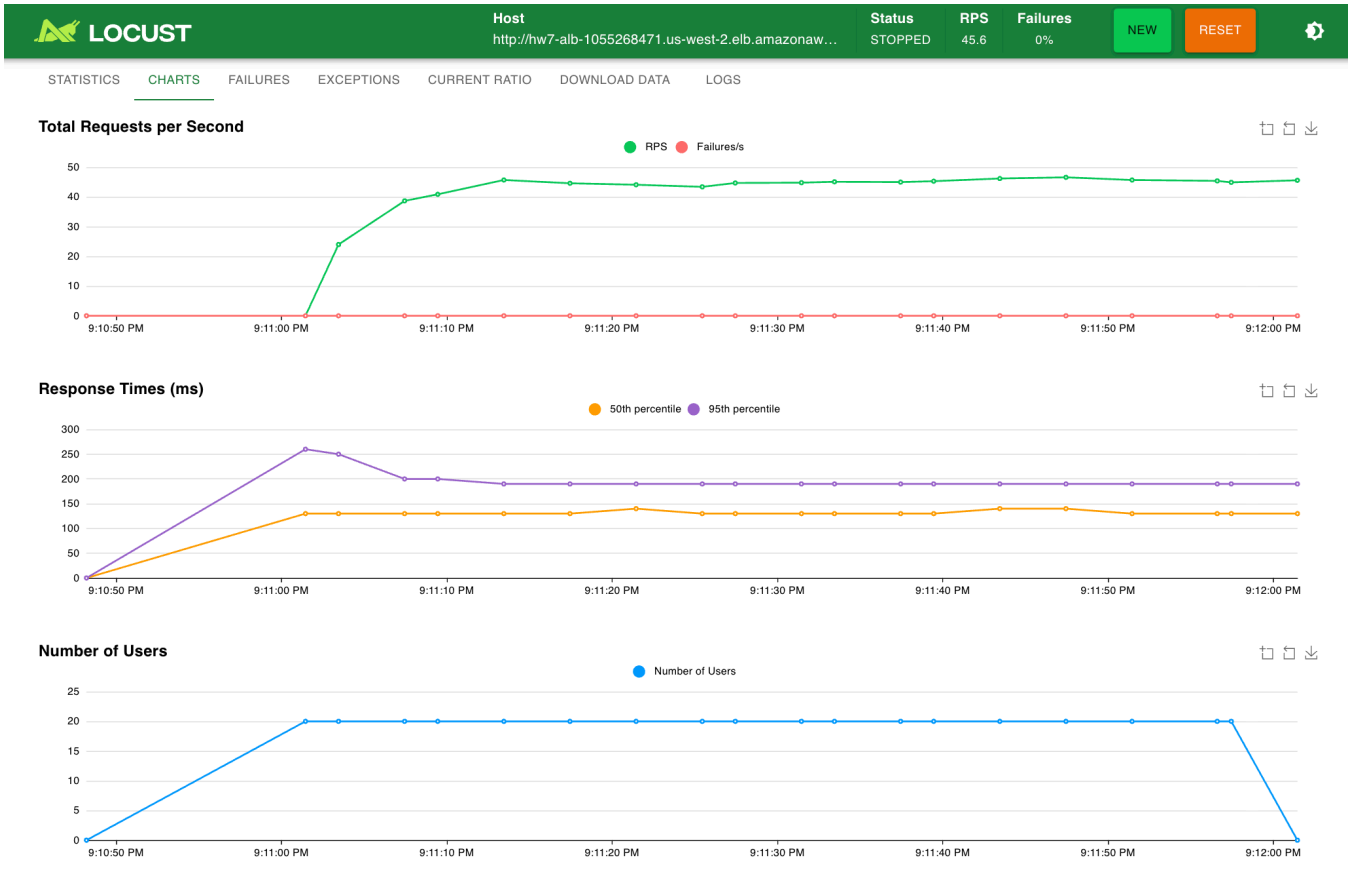




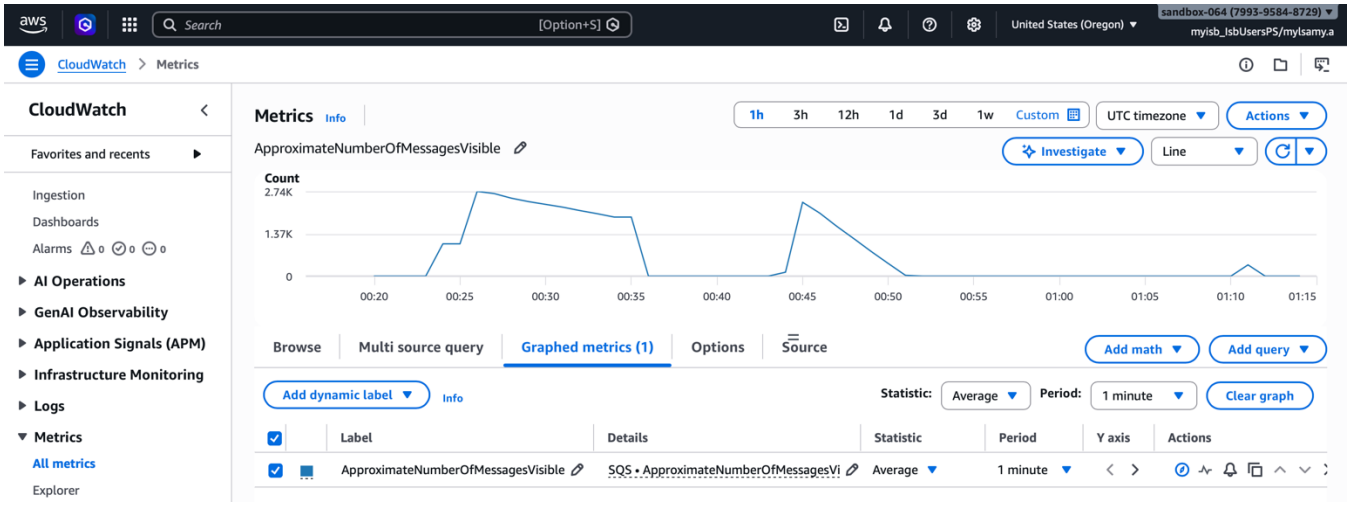
100 Workers



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders/async	2694	0	130	190	270	138.24	87	334	111	45.6	0
	Aggregated	2694	0	130	190	270	138.24	87	334	111	45.6	0



Comparison



Workers	Processing Rate	Peak Queue Depth	Drain Time	RPS	Failures
1	0.33/s	~5,310	~267 min	46.3	0%
5	1.67/s	~2,740	~27 min	50.8	0%
20	6.67/s	~2,400	~7 min	48	0%
100	33.3/s	~300	~2 min	45.6	0%

Minimum workers to prevent queue buildup at 60 orders/s:
60 orders/s × 3s/order = 180 goroutines needed to match ingestion rate.

Analysis

How many times more orders did async accept vs sync?
Async accepted ~46 RPS vs sync's ~2 RPS under flash sale conditions, approximately 23x more orders per second. Under the flash sale, sync processed only 15 successful orders out of 110 attempts (86% failure), while async accepted all 2,723 requests with 0% failure.

What causes queue buildup and how do you prevent it?
Queue builds when the producer rate exceeds the consumer rate. Here, orders arrive at ~46/s but a single worker processes only 0.33/s. Prevention strategies: scale workers until processing rate matches ingestion rate (180 goroutines for 60 orders/s), implement auto-scaling tied to ApproximateNumberOfMessagesVisible, or rate-limit ingestion.

When would you choose sync vs async?

- **Sync**: when the caller needs the result immediately (auth checks, read queries, anything requiring an instant answer). Simpler to reason about, errors surface directly to the caller.
- **Async**: when work can be deferred, the caller only needs acknowledgment, and throughput matters more than latency (order processing, email sends, image resizing, report generation).

Resource utilization: Resource utilization remained stable across all worker counts, ECS tasks stayed within the allocated 256 CPU units and 512MB memory. Since goroutines are lightweight and the workload is I/O-bound (sleeping to simulate payment delay), increasing goroutines from 5 to 100 did not significantly increase CPU or memory pressure.

PART III – SERVERLESS WITH LAMBDA

Instead of managing SQS queues and ECS workers, Lambda subscribes directly to the SNS topic. AWS handles all scaling and infrastructure.

Infrastructure

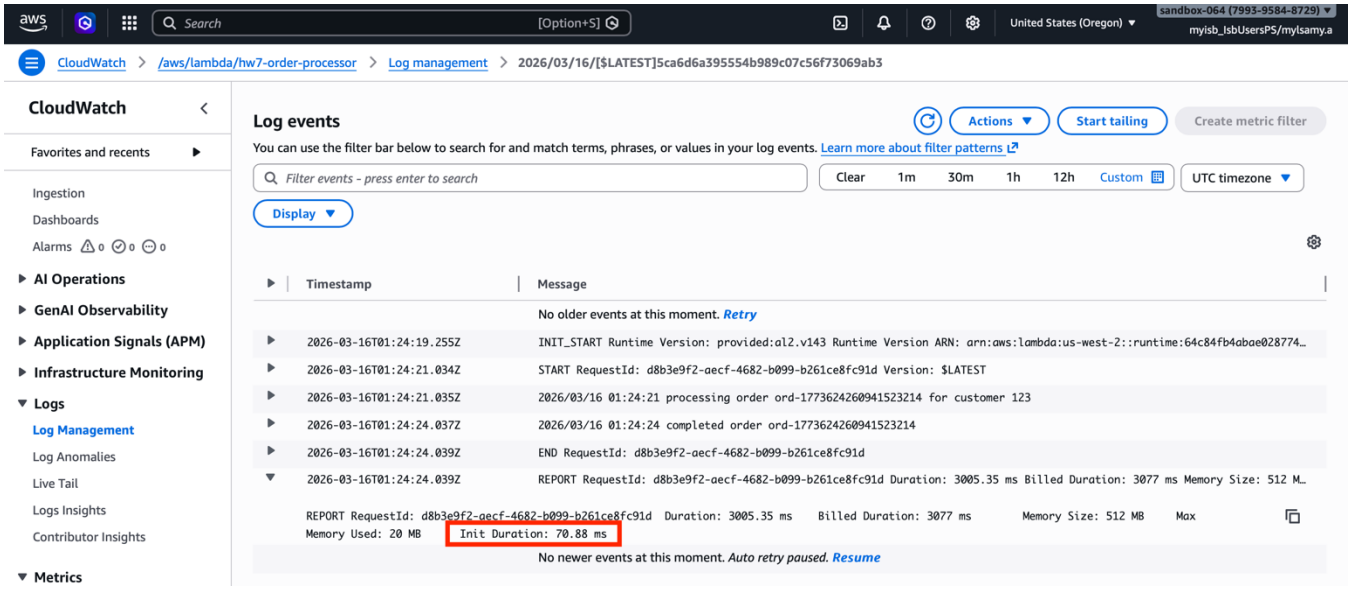
- **Lambda**: hw7-order-processor, 512MB memory, provided.al2 runtime, 30s timeout
- **Trigger**: SNS topic order-processing-events (direct subscription, no SQS)

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

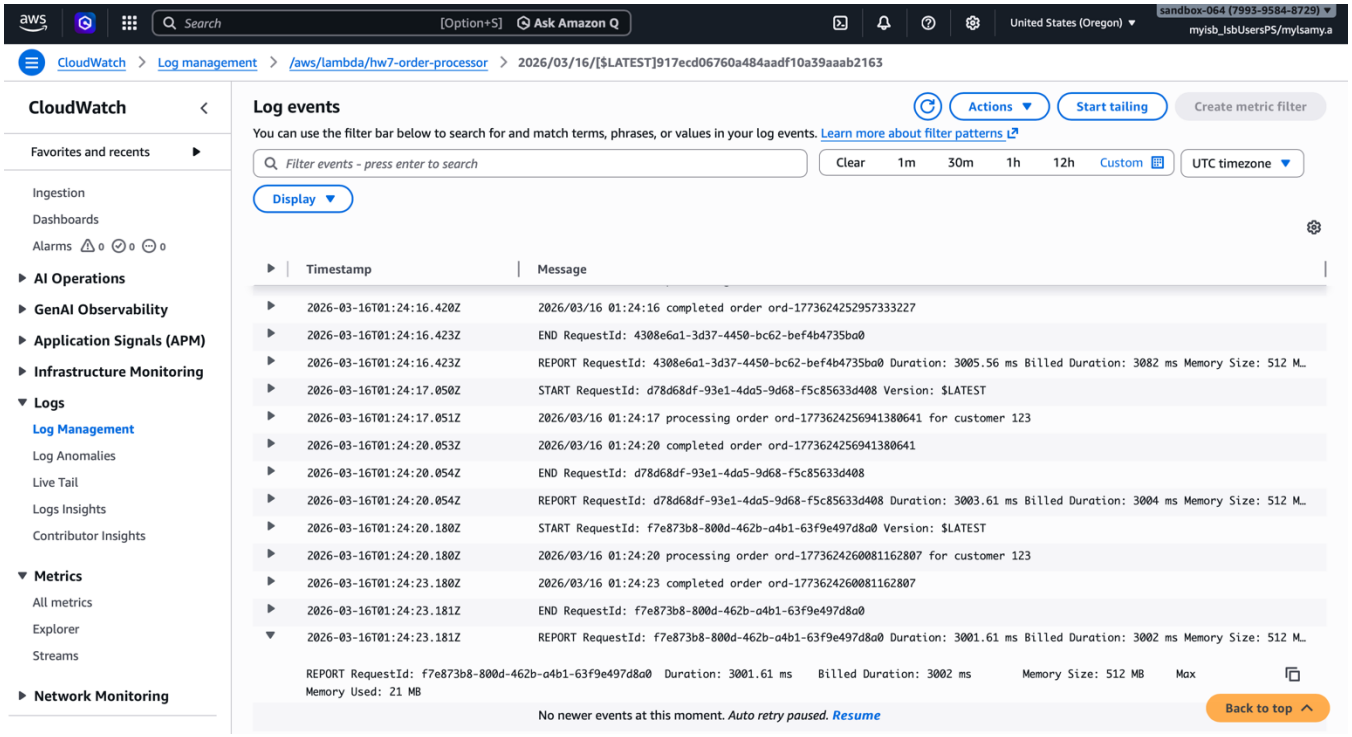
(base) ananyaamylsamy@Ananyaas-MacBook-Pro terraform % clear
(base) ananyaamylsamy@Ananyaas-MacBook-Pro terraform % curl -X POST http://hw7-alb-1055268471.us-west-2.elb.amazonaws.com/orders/async \
  -H "Content-Type: application/json" \
  -d '{"customer_id": 123, "items": [{"product_id": "prod-001", "quantity": 2, "price": 29.99}]}'
{"message":"Order received and queued for processing","order_id":"ord-1773624252957333227","status":"pending"}
(base) ananyaamylsamy@Ananyaas-MacBook-Pro terraform % curl -X POST http://hw7-alb-1055268471.us-west-2.elb.amazonaws.com/orders/async \
  -H "Content-Type: application/json" \
  -d '{"customer_id": 123, "items": [{"product_id": "prod-001", "quantity": 2, "price": 29.99}]}'
{"message":"Order received and queued for processing","order_id":"ord-1773624254824719030","status":"pending"}
(base) ananyaamylsamy@Ananyaas-MacBook-Pro terraform % curl -X POST http://hw7-alb-1055268471.us-west-2.elb.amazonaws.com/orders/async \
  -H "Content-Type: application/json" \
  -d '{"customer_id": 123, "items": [{"product_id": "prod-001", "quantity": 2, "price": 29.99}]}'
{"message":"Order received and queued for processing","order_id":"ord-1773624255880346108","status":"pending"}
(base) ananyaamylsamy@Ananyaas-MacBook-Pro terraform % curl -X POST http://hw7-alb-1055268471.us-west-2.elb.amazonaws.com/orders/async \
  -H "Content-Type: application/json" \
  -d '{"customer_id": 123, "items": [{"product_id": "prod-001", "quantity": 2, "price": 29.99}]}'
{"message":"Order received and queued for processing","order_id":"ord-1773624256941380641","status":"pending"}
(base) ananyaamylsamy@Ananyaas-MacBook-Pro terraform % curl -X POST http://hw7-alb-1055268471.us-west-2.elb.amazonaws.com/orders/async \
  -H "Content-Type: application/json" \
  -d '{"customer_id": 123, "items": [{"product_id": "prod-001", "quantity": 2, "price": 29.99}]}'
{"message":"Order received and queued for processing","order_id":"ord-1773624258094851783","status":"pending"}
(base) ananyaamylsamy@Ananyaas-MacBook-Pro terraform % curl -X POST http://hw7-alb-1055268471.us-west-2.elb.amazonaws.com/orders/async \
  -H "Content-Type: application/json" \
  -d '{"customer_id": 123, "items": [{"product_id": "prod-001", "quantity": 2, "price": 29.99}]}'
{"message":"Order received and queued for processing","order_id":"ord-1773624259111584252","status":"pending"}
(base) ananyaamylsamy@Ananyaas-MacBook-Pro terraform % curl -X POST http://hw7-alb-1055268471.us-west-2.elb.amazonaws.com/orders/async \
  -H "Content-Type: application/json" \
  -d '{"customer_id": 123, "items": [{"product_id": "prod-001", "quantity": 2, "price": 29.99}]}'
{"message":"Order received and queued for processing","order_id":"ord-1773624260081162807","status":"pending"}
(base) ananyaamylsamy@Ananyaas-MacBook-Pro terraform % curl -X POST http://hw7-alb-1055268471.us-west-2.elb.amazonaws.com/orders/async \
  -H "Content-Type: application/json" \
  -d '{"customer_id": 123, "items": [{"product_id": "prod-001", "quantity": 2, "price": 29.99}]}'
{"message":"Order received and queued for processing","order_id":"ord-1773624260941523214","status":"pending"}
(base) ananyaamylsamy@Ananyaas-MacBook-Pro terraform % curl -X POST http://hw7-alb-1055268471.us-west-2.elb.amazonaws.com/orders/async \
  -H "Content-Type: application/json" \
  -d '{"customer_id": 123, "items": [{"product_id": "prod-001", "quantity": 2, "price": 29.99}]}'
{"message":"Order received and queued for processing","order_id":"ord-1773624261920624707","status":"pending"}
(base) ananyaamylsamy@Ananyaas-MacBook-Pro terraform % curl -X POST http://hw7-alb-1055268471.us-west-2.elb.amazonaws.com/orders/async \
  -H "Content-Type: application/json" \
  -d '{"customer_id": 123, "items": [{"product_id": "prod-001", "quantity": 2, "price": 29.99}]}'
{"message":"Order received and queued for processing","order_id":"ord-1773624262885356934","status":"pending"}
(base) ananyaamylsamy@Ananyaas-MacBook-Pro terraform %
```

Cold Start Observations

Cold Start Log



Warm Start Log



Metric	Value
Cold start overhead	70.88ms
Processing time	3005ms
Total cold start billed	3077ms
Cold start impact	70.88 / 3077 = 2.3%
When cold starts occur	First invocation, after ~5+ min idle

Cold start frequency: Cold starts occurred on the first invocation after deployment and would recur after approximately 5+ minutes of idle time. During our test, sending 10 consecutive orders triggered one cold start (the first invocation) and warm starts for all subsequent requests.

Does cold start matter: The 70.88ms cold start overhead is negligible for this workload, it represents only 2.3% of the total 3-second payment processing time. Cold starts would only become a concern for latency-sensitive workloads requiring sub-100ms responses, which is not the case here.

Cost Comparison

Volume	ECS (always-on, 2 tasks)	Lambda
10K orders/month	$17/month	$0 (free tier)
267K orders/month	$17/month	$0 (free tier ceiling)
1.7M orders/month	$17/month	$17/month (break-even)

Free tier: 1M requests + 400K GB-seconds/month For 3s × 0.5GB = 1.5 GB-s per order -> 400K / 1.5 = ~267K orders/month FREE

Switch Recommendation

Based on our observations, the startup should switch to Lambda for order processing. The cold start overhead of 70.88ms is negligible, just 2.3% of the 3-second payment workflow, and has no meaningful impact on customer experience. For our current volume (well under 267K orders/month), Lambda costs $0 compared to $17/month for always-on ECS tasks. The elimination of SQS queue management, worker scaling, queue timeout tuning, and ECS health monitoring removes significant operational burden. The main trade-off is losing SQS durability guarantees, SNS retries only twice before discarding a message, but at startup scale this risk is acceptable given the cost and operational savings. We would revisit this decision if order volume approaches 1.7M/month, at which point ECS becomes cost-competitive and SQS guarantees become more critical.

TEAM COMPARISON REPORT

Part II: Async Order Processing with SNS and SQS

Phase 1: Sync Flash Sale Comparison

Metric	Katie	Manasha	Ananyaa
Requests	26	26	110
Failures	100%	0% (local)	86%
RPS	0.3	0.4	2
Median Response	40000ms	~45000ms	10000ms
95th Percentile	60000ms	~70000ms	10000ms

Observations: All three implementations demonstrated the synchronous bottleneck under flash sale load. The difference in failure rates reflects different semaphore configurations, Katie used a capacity of 1 (strict single processor), resulting in 100% timeouts. Manasha's system showed 0% failures but extremely high latency, meaning customers were not rejected but waited up to 70 seconds. Ananyaa used a capacity of 5 matching normal operations, resulting in 86% failures when 20 users exceeded the slots. All results confirm the same conclusion: synchronous payment processing cannot handle flash sale load.

Phase 2: Bottleneck Analysis Comparison

Metric	Katie	Manasha	Ananyaa
Max throughput	0.33 orders/s	0.33 orders/s	0.33 orders/s
Flash sale demand	20 orders/s	20 orders/s	20 orders/s
Orders lost per second	19.67 orders/s	19.67 orders/s	19.67 orders/s

Observations: All three reached identical conclusions, the payment processor is the fixed bottleneck at 0.33 orders/second regardless of concurrent users.

Phase 3: Async Flash Sale Comparison

Metric	Katie	Manasha	Ananyaa
Requests	5809	3872	2723
Failures	0%	0%	0%
RPS	75.7	49.9	46.3
Median Response	200ms	~102ms	120ms

Observations: All three achieved 0% failures, the async architecture solved the flash sale problem in every implementation. RPS differences reflect test duration and spawn rate variations rather than implementation differences. The core result is consistent: async decouples acceptance from processing, eliminating timeouts entirely.

Phase 4: Queue Buildup Comparison

Metric	Katie	Manasha	Ananyaa
Peak queue depth	~925	~3,840	~5,310
Queue growth rate	~59.67 msg/s	~49.67 msg/s	~45.7 msg/s
Time to drain (1 worker)	~46 min	~192 min	~267 min

Observations: All three observed significant queue buildup with 1 worker. Differences in peak depth and drain time reflect different test durations and acceptance rates. The core finding is consistent: a single worker at 0.33 orders/second cannot keep up with flash sale traffic.

Phase 5: Worker Scaling Comparison

Workers	Katie Peak Depth	Manasha Peak Depth	Ananyaa Peak Depth
5	~5,400	~3,870	~2,740
20	~3,600	~2,690	~2,400
100	~2,300	~900	~300

Workers	Katie Drain Time	Manasha Drain Time	Ananyaa Drain Time
5	~54 min	~38 min	~27 min
20	~9 min	~10 min	~7 min
100	~1 min	~2 min	~2 min

Observations: All three show the same trend, more workers means lower peak queue depth and faster drain time. Manasha calculated 150 workers needed at 50 orders/second, Ananyaa calculated 180 workers at 60 orders/second, both derived from the same formula: workers = acceptance_rate × processing_time, with the difference explained by different measured acceptance rates.

PART III: LAMBDA COMPARISON

Metric	Katie	Manasha	Ananyaa
Cold start overhead	~73ms avg	76ms–1284ms	70.88ms
Warm start overhead	0ms	0ms	0ms
Cold start impact	~2.4%	2.5%–42%	2.3%
Break-even volume	1.7M orders/month	1.7M orders/month	1.7M orders/month

Observations: Katie and Ananyaa sent orders sequentially, getting one cold start followed by warm starts. Manasha sent 10 orders simultaneously, triggering 10 concurrent cold starts with high variance (76ms–1284ms). This reveals an important production consideration: concurrent cold starts compete for resources and show much higher variance than sequential ones. All three reached the same break-even calculation of ~267K orders/month free and agreed Lambda is the right choice at startup scale.

Conclusion

1. **Sync breaks under load regardless of implementation**, all three saw the same failure pattern at 20 concurrent users
2. **Async solves acceptance rate**, 0% failures across all three implementations
3. **Queue buildup is universal**, all three experienced it with 1 worker; all three solved it by scaling workers
4. **Lambda cold starts are predictable sequentially but variable concurrently**, concurrent testing reveals variance that sequential testing misses
5. **Cost math is identical**, all three reached the same break-even calculation of ~267K orders/month free